Chapter 8
Cluster Analysis: Basic Concepts and Methods

Outline

- Cluster analysis
- Partitioning methods
- Hierarchical methods
- Density-based and grid-based methods
- Evaluation of clustering

Outline

- Cluster analysis
  - What is cluster analysis?
  - Requirements for cluster analysis
  - Overview of basic clustering methods
- Partitioning methods
- Hierarchical methods
- Density-based and grid-based methods
- Evaluation of clustering

Cluster Analysis: A Quick Overview

- When flying over a city, one can easily identify fields, forests, commercial areas, and residential areas based on their features, without anyone's explicit "training"—This is the power of cluster analysis
- This chapter and the next systematically study cluster analysis methods and help answer the following:
  - What are the different proximity measures for effective clustering?
  - Can we cluster a massive number of data points efficiently?
  - Can we find clusters of arbitrary shape? At multiple levels of granularity?
  - How can we judge the quality of the clusters discovered by our system?

The Value of Cluster Analysis

- What is the value of cluster analysis?
  - Cluster analysis helps you partition massive data into groups based on its features
  - Cluster analysis will often help subsequent data mining processes such as pattern discovery, classification, and outlier analysis
- What roles does cluster analysis play in the Data Mining Specialization?
  - You will learn various scalable methods to find clusters from massive data
  - You will learn how to mine different kinds of clusters effectively
  - You will also learn how to evaluate the quality of the clusters you find
  - Cluster analysis will help with classification, outlier analysis, and other data mining tasks

Broad Applications of Cluster Analysis

- Data summarization, compression, and reduction
  - Examples: Image processing or vector quantization
- Collaborative filtering, recommendation systems, or customer segmentation
  - Finding like-minded users or similar products
- Dynamic trend detection
  - Clustering stream data and detecting trends and patterns
- Multimedia data analysis, biological data analysis, and social network analysis
  - Examples: Clustering video/audio clips or gene/protein sequences
- A key intermediate step for other data mining tasks
  - Generating a compact summary of data for classification, pattern discovery, and hypothesis generation and testing
  - Outlier detection: Outliers are those “far away” from any cluster

What Is Cluster Analysis?

- What is a cluster?
  - A cluster is a collection of data objects which are
    - Similar (or related) to one another within the same group (i.e., cluster)
    - Dissimilar (or unrelated) to the objects in other groups (i.e., clusters)
- Cluster analysis (or clustering, data segmentation, …)
  - Given a set of data points, partition them into a set of groups (i.e., clusters) which are as similar as possible
- Cluster analysis is unsupervised learning (i.e., no predefined classes)
  - This contrasts with classification (i.e., supervised learning)
- Typical ways to use/apply cluster analysis
  - As a stand-alone tool to get insight into data distribution, or
  - As a preprocessing (or intermediate) step for other algorithms

Cluster Analysis: Applications

- A key intermediate step for other data mining tasks
  - Generating a compact summary of data for classification, pattern discovery, hypothesis generation and testing, etc.
  - Outlier detection: Outliers—those “far away” from any cluster
- Data summarization, compression, and reduction
  - Ex. Image processing: Vector quantization
- Collaborative filtering, recommendation systems, or customer segmentation
  - Find like-minded users or similar products
- Dynamic trend detection
  - Clustering stream data and detecting trends and patterns
- Multimedia data analysis, biological data analysis and social network analysis
  - Ex. Clustering images or video/audio clips, gene/protein sequences, etc.

Considerations for Cluster Analysis

- Partitioning criteria
  - Single level vs. hierarchical partitioning (often, multi-level hierarchical partitioning is desirable, e.g., grouping topical terms)
- Separation of clusters
  - Exclusive (e.g., one customer belongs to only one region) vs. non-exclusive (e.g., one document may belong to more than one class)
- Similarity measure
  - Distance-based (e.g., Euclidean, road network, vector) vs. connectivity-based (e.g., density or contiguity)
- Clustering space
  - Full space (often when low dimensional) vs. subspaces (often in high-dimensional clustering)

Requirements and Challenges

- Quality
  - Ability to deal with different types of attributes: Numerical, categorical, text, multimedia, networks, and mixture of multiple types
  - Discovery of clusters with arbitrary shape
  - Ability to deal with noisy data
- Scalability
  - Clustering all the data instead of only on samples
  - High dimensionality
  - Incremental or stream clustering and insensitivity to input order
- Constraint-based clustering
  - User-given preferences or constraints; domain knowledge; user queries
- Interpretability and usability

Cluster Analysis: A Multi-Dimensional Categorization

- Technique-Centered
  - Distance-based methods
  - Density-based and grid-based methods
  - Probabilistic and generative models
  - Leveraging dimensionality reduction methods
  - High-dimensional clustering
  - Scalable techniques for cluster analysis
- Data Type-Centered
  - Clustering numerical data, categorical data, text data, multimedia data, time-series data, sequences, stream data, networked data, uncertain data
- Additional Insight-Centered
  - Visual insights, semi-supervised, ensemble-based, validation-based

Typical Clustering Methodologies

- Distance-based methods
  - Partitioning algorithms: K-Means, K-Medians, K-Medoids
  - Hierarchical algorithms: Agglomerative vs. divisive methods
- Density-based and grid-based methods
  - Density-based: Data space is explored at a high-level of granularity and then post-processing to put together dense regions into an arbitrary shape
  - Grid-based: Individual regions of the data space are formed into a grid-like structure
- Probabilistic and generative models: Modeling data from a generative process
  - Assume a specific form of the generative model (e.g., mixture of Gaussians)
  - Model parameters are estimated with the Expectation-Maximization (EM) algorithm (using the available dataset, for a maximum likelihood fit)
  - Then estimate the generative probability of the underlying data points
- High-dimensional clustering

High-dimensional Dlustering

- Subspace clustering: Find clusters on various subspaces
  - Bottom-up, top-down, correlation-based methods vs. δ-cluster methods
- Dimensionality reduction: A vertical form (i.e., columns) of clustering
  - Columns are clustered; may cluster rows and columns together (co-clustering)
  - Probabilistic latent semantic indexing (PLSI) then LDA: Topic modeling of text data
    - A cluster (i.e., topic) is associated with a set of words (i.e., dimensions) and a set of documents (i.e., rows) simultaneously
  - Nonnegative matrix factorization (NMF) (as one kind of co-clustering)
    - A nonnegative matrix A (e.g., word frequencies in documents) can be approximately factorized two non-negative low rank matrices U and V
  - Spectral clustering: Use the spectrum of the similarity matrix of the data to perform dimensionality reduction for clustering in fewer dimensions

Clustering Different Types of Data (I)

- Numerical data
  - Most earliest clustering algorithms were designed for numerical data
- Categorical data (including binary data)
  - Discrete data, no natural order (e.g., sex, race, zip-code, and market-basket)
- Text data: Popular in social media, Web, and social networks
  - Features: High-dimensional, sparse, value corresponding to word frequencies
  - Methods: Combination of k-means and agglomerative; topic modeling; co-clustering
- Multimedia data: Image, audio, video (e.g., on Flickr, YouTube)
  - Multi-modal (often combined with text data)
  - Contextual: Containing both behavioral and contextual attributes
    - Images: Position of a pixel represents its context, value represents its behavior
    - Video and music data: Temporal ordering of records represents its meaning

Clustering Different Types of Data (II)

- Time-series data: Sensor data, stock markets, temporal tracking, forecasting, etc.
  - Data are temporally dependent
  - Time: contextual attribute; data value: behavioral attribute
  - Correlation-based online analysis (e.g., online clustering of stock to find stock tickers)
  - Shape-based offline analysis (e.g., cluster ECG based on overall shapes)
- Sequence data: Weblogs, biological sequences, system command sequences
  - Contextual attribute: Placement (rather than time)
  - Similarity functions: Hamming distance, edit distance, longest common subsequence
  - Sequence clustering: Suffix tree; generative model (e.g., Hidden Markov Model)
- Stream data:
  - Real-time, evolution and concept drift, single pass algorithm
  - Create efficient intermediate representation, e.g., micro-clustering

Clustering Different Types of Data (III)

- Graphs and homogeneous networks
  - Every kind of data can be represented as a graph with similarity values as edges
  - Methods: Generative models; combinatorial algorithms (graph cuts); spectral methods; non-negative matrix factorization methods
- Heterogeneous networks
  - A network consists of multiple typed nodes and edges (e.g., bibliographical data)
  - Clustering different typed nodes/links together (e.g., NetClus)
- Uncertain data: Noise, approximate values, multiple possible values
  - Incorporation of probabilistic information will improve the quality of clustering
- Big data: Model systems may store and process very big data (e.g., weblogs)
  - Ex. Google's MapReduce framework
    - Use Map function to distribute the computation across different machines
    - Use Reduce function to aggregate results obtained from the Map step

User Insights and Interactions in Clustering

- Visual insights: One picture is worth a thousand words
  - Human eyes: High-speed processor linking with a rich knowledge-base
  - A human can provide intuitive insights; HD-eye: visualizing HD clusters
- Semi-supervised insights: Passing user's insights or intention to system
  - User-seeding: A user provides a number of labeled examples, approximately representing categories of interest
- Multi-view and ensemble-based insights
  - Multi-view clustering: Multiple clusterings represent different perspectives
  - Multiple clustering results can be ensembled to provide a more robust solution
- Validation-based insights: Evaluation of the quality of clusters generated
  - May use case studies, specific measures, or pre-existing labels

Outline

- Cluster analysis
- Partitioning methods
  - K-means: a centroid-based techniques
  - Variations of k-means
- Hierarchical methods
- Density-based and grid-based methods
- Evaluation of clustering

Partitioning Algorithms: Basic Concepts

- Partitioning method: Discovering the groupings in the data by optimizing a specific objective function and iteratively improving the quality of partitions
- K-partitioning method: Partitioning a dataset D of n objects into a set of K clusters so that an objective function is optimized (e.g., the sum of squared distances is minimized, where c_k is the centroid or medoid of cluster C_k)
  - A typical objective function: Sum of Squared Errors (SSE)

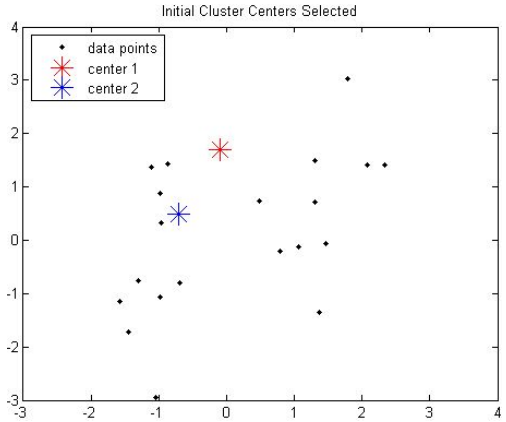
$$SSE(C) = \sum_{k=1}^{K} \sum_{x_i \in C_k} |x_i - c_k|^2$$

- Problem definition: Given K, find a partition of K clusters that optimizes the chosen partitioning criterion
  - Global optimal: Needs to exhaustively enumerate all partitions
  - Heuristic methods (i.e., greedy algorithms): K-Means, K-Medians, K-Medoids, etc.

The K-Means Clustering Method

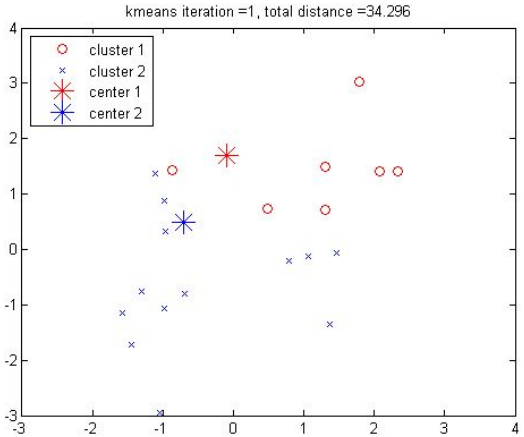
- K-Means (MacQueen'67, Lloyd'57/'82)
  - Each cluster is represented by the center of the cluster
- Given K, the number of clusters, the K-Means clustering algorithm is outlined as follows
    - Select K points as initial centroids
    - Repeat
      - Form K clusters by assigning each point to its closest centroid
      - Re-compute the centroids (i.e., mean point) of each cluster
    - Until convergence criterion is satisfied
- Different kinds of measures can be used
  - Manhattan distance (L1 norm), Euclidean distance (L2 norm), Cosine similarity

Example: K-Means Clustering

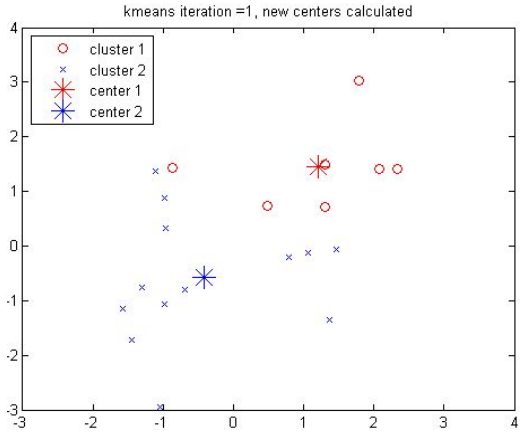


The original data points & randomly select K = 2 centroids

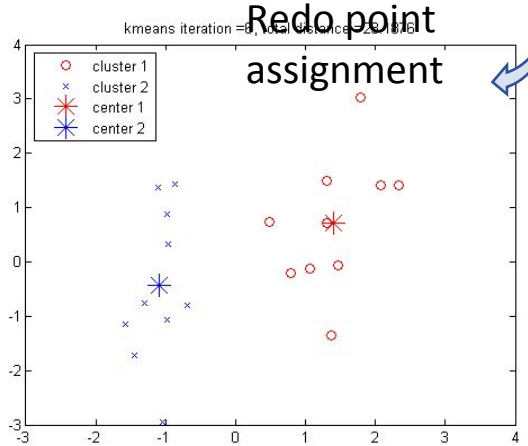
Assign points to clusters



Recompute cluster centers



Redo point assignment



Execution of the K-Means Clustering Algorithm

Select K points as initial centroids

Repeat

- Form K clusters by assigning each point to its closest centroid
- Re-compute the centroids (i.e., *mean point*) of each cluster

Until convergence criterion is satisfied

Discussion on the K-Means Method

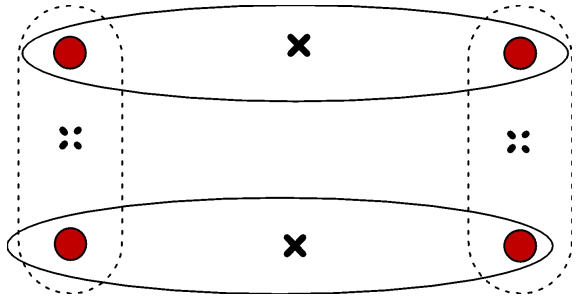
- Efficiency: O(tKn) where n: # of objects, K: # of clusters, and t: # of iterations
  - Normally, K, t << n; thus, an efficient method
- K-means clustering often terminates at a local optimal
  - Initialization can be important to find high-quality clusters
- Need to specify K, the number of clusters, in advance
  - There are ways to automatically determine the “best” K
  - In practice, one often runs a range of values and selected the “best” K value
- Sensitive to noisy data and outliers
  - Variations: Using K-medians, K-medoids, etc.
- K-means is applicable only to objects in a continuous n-dimensional space
  - Using the K-modes for categorical data
- Not suitable to discover clusters with non-convex shapes
  - Using density-based clustering, kernel K-means, etc.

Variations of K-Means

- Choosing better initial centroid estimates
  - K-means++, Intelligent K-Means, Genetic K-Means
- Choosing different representative prototypes for the clusters
  - K-Medoids, K-Medians, K-Modes
- Applying feature transformation techniques
  - Weighted K-Means, Kernel K-Means

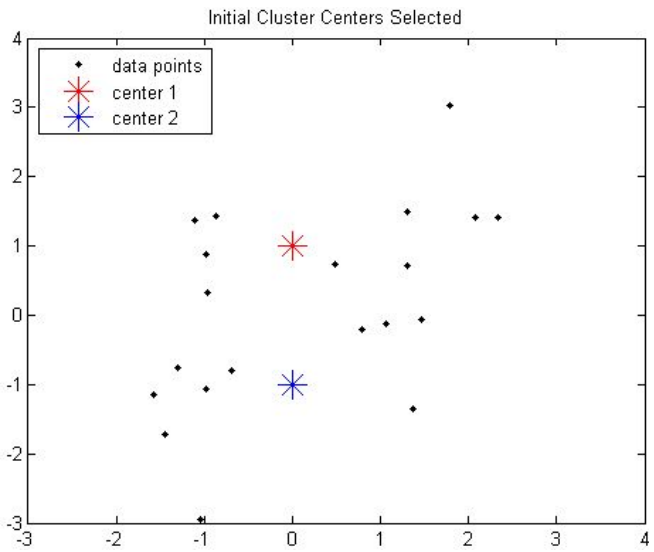
Initialization of K-Means

- Different initializations may generate rather different clustering results (some could be far from optimal)
- Original proposal (MacQueen'67): Select K seeds randomly
  - Need to run the algorithm multiple times using different seeds
- There are many methods proposed for better initialization of *k* seeds

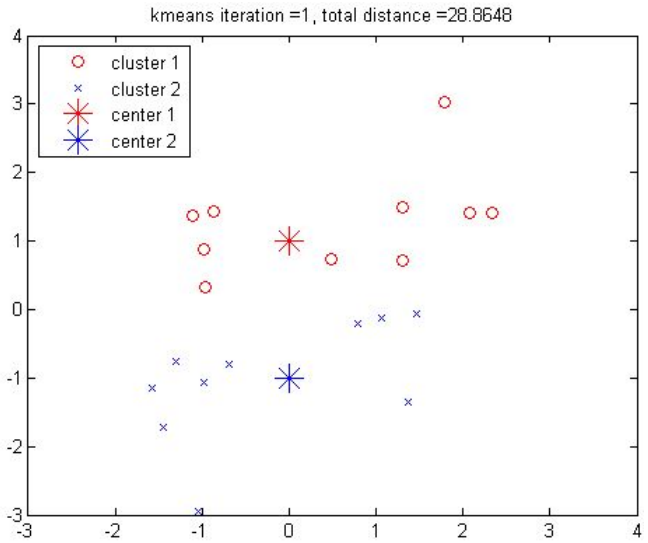


- ***K-Means*++** (Arthur & Vassilvitskii'07):
  - The first centroid is selected at random
  - The next centroid selected is the one that is farthest from the currently selected (selection is based on a weighted probability score)
  - The selection continues until *K* centroids are obtained

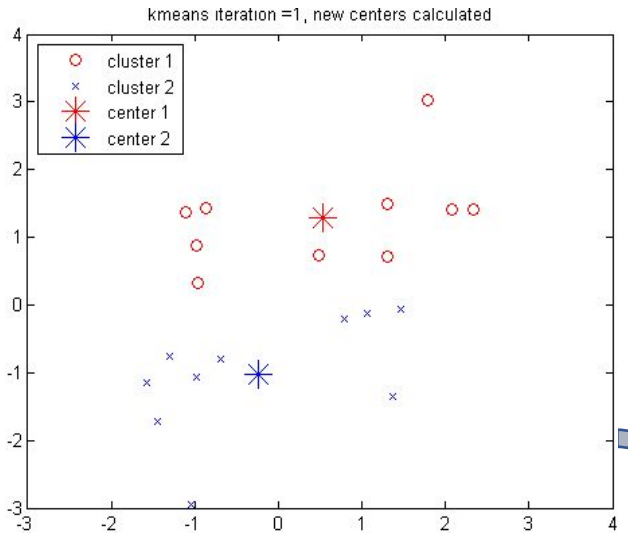
Example: Poor Initialization May Lead to Poor

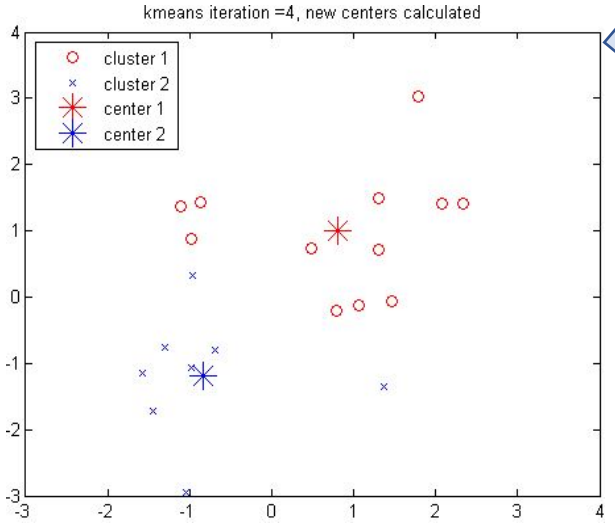


Assign points to clusters



Recompute cluster centers





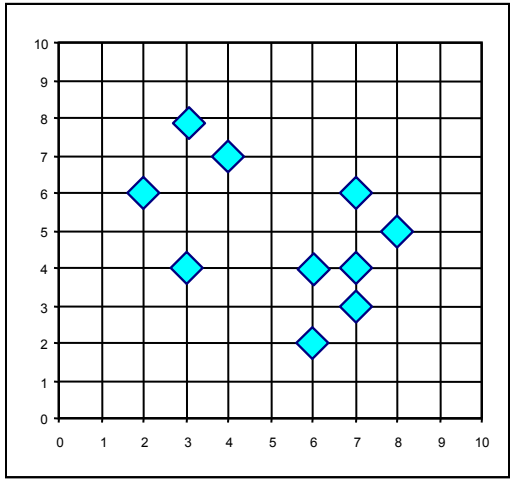
Another random selection of k centroids for the same data points

- Rerun of the *K-Means* using another random *K* seeds
- This run of *K*-Means generates a poor quality clustering

Handling Outliers: From K-Means to K-Medoids

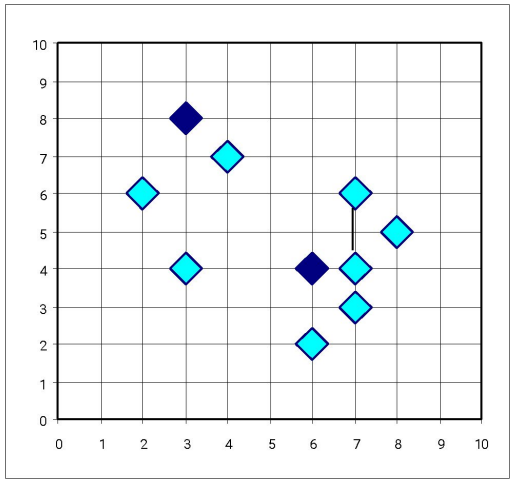
- The K-Means algorithm is sensitive to outliers
  - An object with an extremely large value may substantially distort the distribution of the data
- K-Medoids: Instead of taking the mean value of the object in a cluster as a reference point, medoids can be used, which is the most centrally located object in a cluster
- The K-Medoids clustering algorithm:
  - Select K points as the initial representative objects (i.e., as initial K medoids)
  - Repeat
    - Assigning each point to the cluster with the closest medoid
    - Randomly select a non-representative object o_i
    - Compute the total cost S of swapping the medoid m with o_i
    - If S < 0, then swap m with o_i to form the new set of medoids
  - Until convergence criterion is satisfied

PAM: A Typical K-Medoids Algorithm

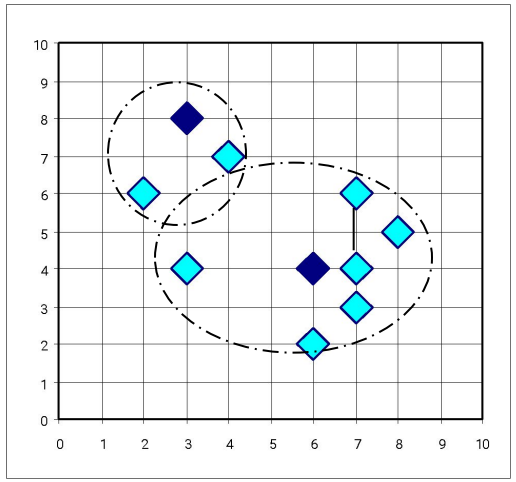


$K = 2$

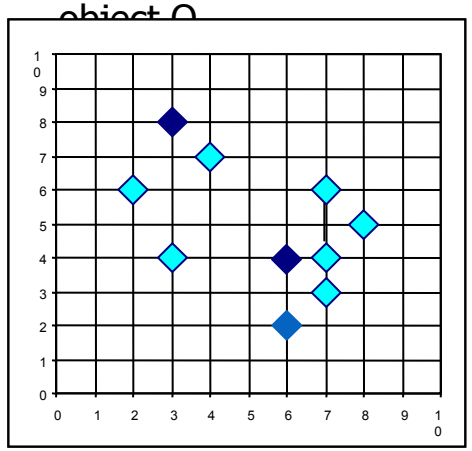
Arbitrary choose K object as initial medoids



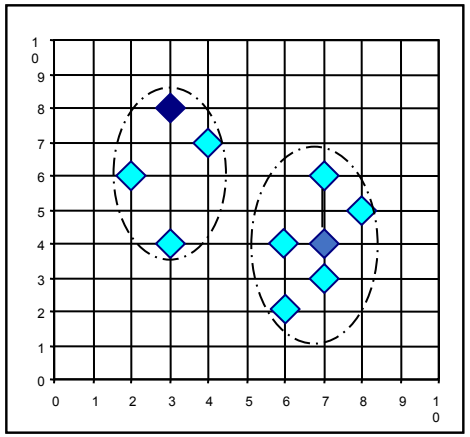
Assign each remaining object to nearest medoids



Randomly select a non-medoid object O



Compute total cost of swapping



Swapping O and O_{ramdom}

If quality is improved

Select initial *K medoids* randomly

Repeat

Object re-assignment

Swap medoid *m* with o_i if it improves the clustering quality

Until convergence criterion is satisfied

Discussion on K-Medoids Clustering

- K-Medoids Clustering: Find representative objects (medoids) in clusters
- PAM (Partitioning Around Medoids: Kaufmann & Rousseeuw 1987)
  - Starts from an initial set of medoids, and
  - Iteratively replaces one of the medoids by one of the non-medoids if it improves the total sum of the squared errors (SSE) of the resulting clustering
  - PAM works effectively for small data sets but does not scale well for large data sets (due to the computational complexity)
  - Computational complexity: PAM: $O(K(n - K)2)$ (quite expensive!)
- Efficiency improvements on PAM
  - CLARA (Kaufmann & Rousseeuw, 1990):
    - PAM on samples; $O(Ks2 + K(n - K))$, s is the sample size
  - CLARANS (Ng & Han, 1994): Randomized re-sampling, ensuring efficiency + quality

K-Medians: Handling Outliers by Computing Medians

- Medians are less sensitive to outliers than means
  - Think of the median salary vs. mean salary of a large firm when adding a few top executives!
- K-Medians: Instead of taking the mean value of the object in a cluster as a reference point, medians are used (L1-norm as the distance measure)
- The criterion function for the K-Medians algorithm

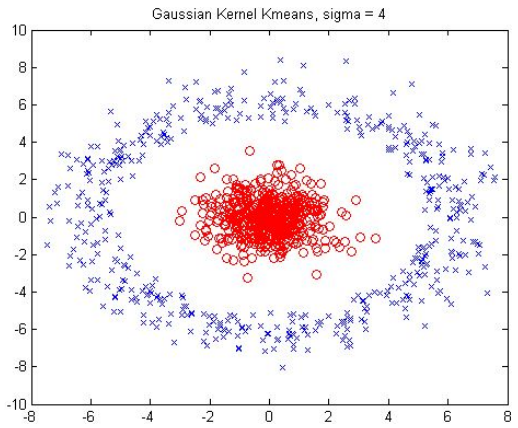
$$S = \sum_{k=1}^{K} \sum_{x_i \in C_k} |x_{ij} - med_{kj}|$$

- The K-Medians clustering algorithm:
  - Select K points as the initial representative objects (i.e., as initial K medians)
  - Repeat
    - Assign every point to its nearest median
    - Re-compute the median using the median of each individual feature
  - Until convergence criterion is satisfied

K-Modes: Clustering Categorical Data

- K-Means cannot handle non-numerical (categorical) data
  - Mapping categorical value to 1/0 cannot generate quality clusters for high-dimensional data
- K-Modes: An extension to K-Means by replacing means of clusters with modes
- Dissimilarity measure between object X and the center of a cluster Z
  - $\Phi(x_j, z_j) = 1$ when $x_j \neq z_j$; and 0 otherwise
- This dissimilarity measure (distance function) is frequency-based
- Algorithm is still based on iterative object cluster assignment and centroid update
- A fuzzy K-Modes method is proposed to calculate a fuzzy cluster membership value for each object to each cluster
- A mixture of categorical and numerical data: Using a K-Prototype method

Kernel K-Means Clustering



- Kernel K-Means can be used to detect non-convex clusters
  - K-Means can only detect clusters that are linearly separable
- Idea: Project data onto the high-dimensional kernel space, and then perform K-Means clustering
  - Map data points in the input space onto a high-dimensional feature space using the kernel function
  - Perform K-Means on the mapped feature space
- Computational complexity is higher than K-Means
  - Need to compute and store n x n kernel matrix generated from the kernel function on the original data
- The widely studied spectral clustering can be considered as a variant of Kernel K-Means clustering

Kernel Functions and Kernel K-Means Clustering

- Typical kernel functions:
  - Polynomial kernel of degree h: $K(\mathbf{X}_i, \mathbf{X}_j) = (\mathbf{X}_i \mathbf{X}_j + 1)^h$
  - Gaussian radial basis function (RBF) kernel: $K(\mathbf{X}_i, \mathbf{X}_j) = e^{-\frac{|\mathbf{X}_i - \mathbf{X}_j|^2}{2\sigma^2}}$
  - Sigmoid kernel: $K(\mathbf{X}_i, \mathbf{X}_j) = \tanh(\kappa \mathbf{X}_i \cdot \mathbf{X}_j - \delta)$
- The formula for kernel matrix K for any two points $\mathbf{X}_i, \mathbf{X}_j \in C_k$ is $K(\mathbf{X}_i, \mathbf{X}_j) = \phi(\mathbf{X}_i) \cdot \phi(\mathbf{X}_j)$
- The SSE criterion of kernel K-means: $SSE(C) = \sum_{k=1}^{K} \sum_{x_i \in C_k} |\phi(x_i) - c_k|^2$
  - The formula for the cluster centroid: $c_k = \frac{\sum_{x_i \in C_k} \phi(x_i)}{|C_k|}$
- Clustering can be performed without the actual individual projections $\phi(x_i)$ and $\phi(x_j)$ for the data points $x_i, x_j \in C_k$

Example: Kernel Functions and Kernel K-Means Clustering

-
- Gaussian radial basis function (RBF) kernel: $K\left(\mathbf{X}_i, \mathbf{X}_j\right) = e^{-\frac{\left|\mathbf{x}_i - \mathbf{x}_j\right|^2}{2\sigma^2}}$
- Suppose there are 5 original 2-dimensional points: $x_1(0,0), x_2(4,4), x_3(-4,4), x_4(-4,-4), x_5(4,-4)$

Example: Kernel Functions and Kernel K-Means Clustering

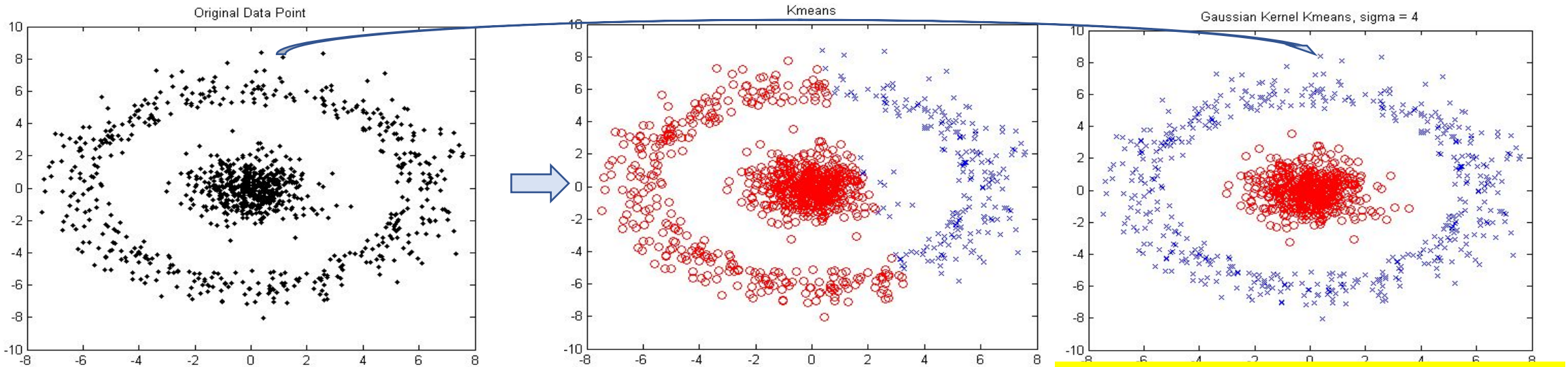
- If we set σ to 4, we will have the following points in the kernel space
  - E.g., $||x_1 - x_2||^2 = (0-4)^2 + (0-4)^2 = 32$, therefore, $K(x_1, x_2) = e^{-\frac{32}{2 \cdot 4^2}} = e^{-1}$

Original Space

x_1		
x_2		
x_3		
x_4		
x_5		

RBF Kernel Space ($\sigma = 4$)

Example: Kernel K-Means Clustering



The original data set

The result of K-Means clustering

The result of Gaussian Kernel K-Means clustering

- The above data set cannot generate quality clusters by K-Means since it contains non-covex clusters
- Gaussian RBF Kernel transformation maps data to a kernel matrix K for any two points x_i, x_j: $K_{x_i x_j} = \phi(x_i) \bullet \phi(x_j)$ and Gaussian kernel: $K(\boldsymbol{X}_i, \boldsymbol{X}_j) = e^{-\|X_i - X_j\|^2 / 2\sigma^2}$
- K-Means clustering is conducted on the mapped data, generating quality clusters

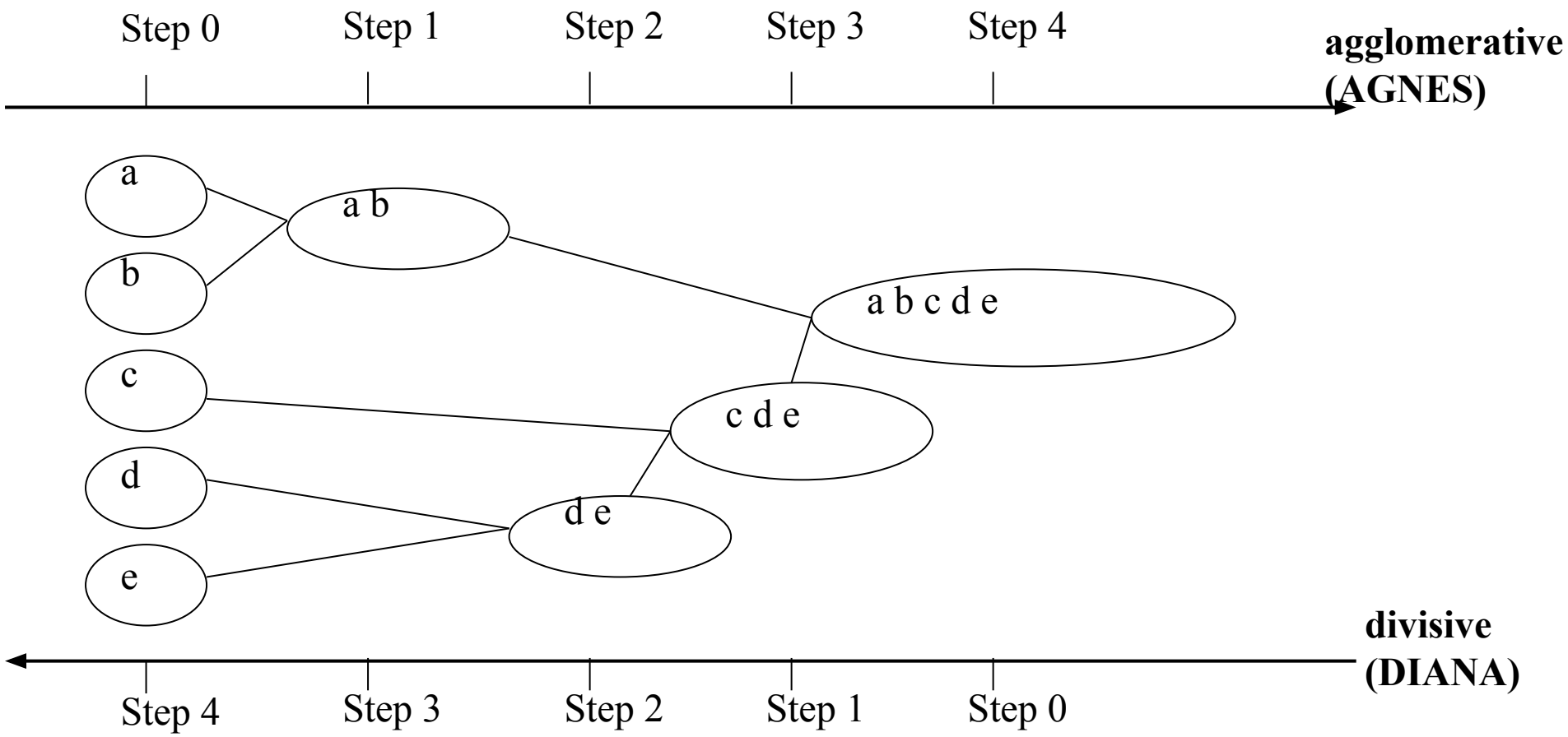
Outline

- Cluster analysis
- Partitioning methods
- Hierarchical methods
  - Basic concepts of hierarchical clustering
  - Agglomerative hierarchical clustering
  - Divisive hierarchical clustering
  - BIRCH: scalable hierarchical clustering using clustering feature trees
  - Probabilistic hierarchical clustering
- Density-based and grid-based methods
- Evaluation of clustering

Hierarchical Clustering: Basic Concepts

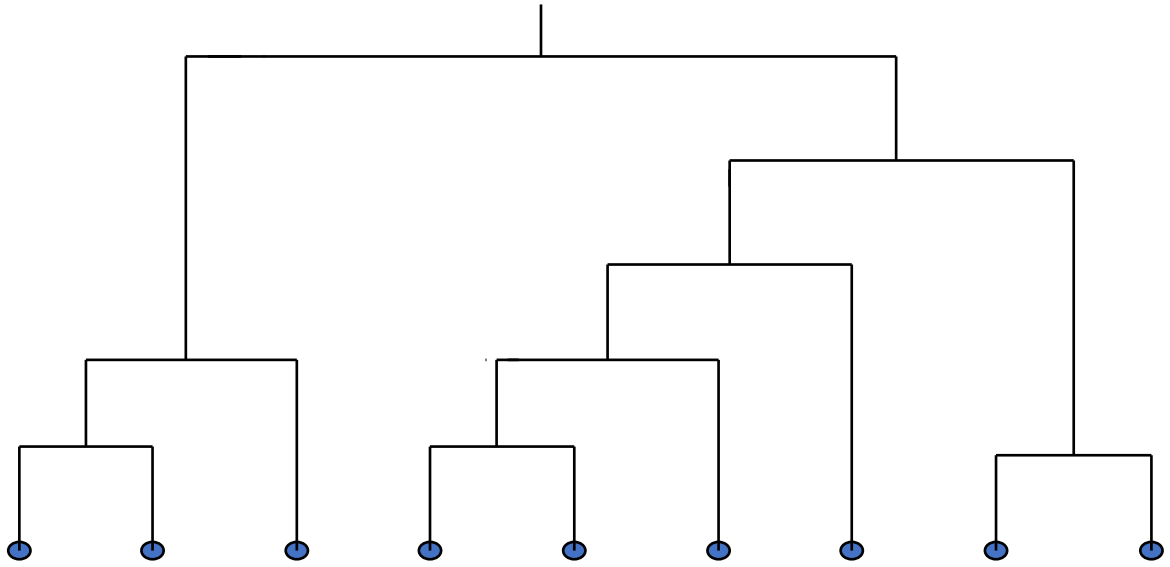
- Hierarchical clustering
  - Generate a clustering hierarchy (drawn as a dendrogram)
  - Not required to specify K, the number of clusters
  - More deterministic
  - No iterative refinement
- Two categories of algorithms
  - Agglomerative: Start with singleton clusters, continuously merge two clusters at a time to build a bottom-up hierarchy of clusters
  - Divisive: Start with a huge macro-cluster, split it continuously into two groups, generating a top-down hierarchy of clusters

Agglomerative vs. Divisive Clustering



Dendrogram: How Clusters are Merged

- Dendrogram: Decompose a set of data objects into a tree of clusters by multi-level nested partitioning
- A clustering of the data objects is obtained by cutting the dendrogram at the desired level, then each connected component forms a cluster

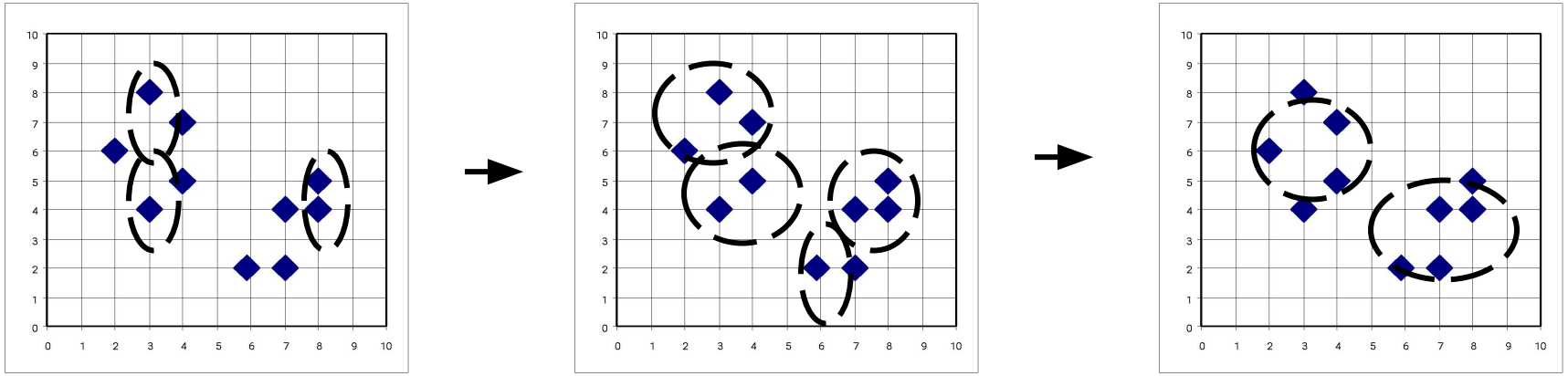


Hierarchical clustering generates a dendrogram (a hierarchy of clusters)

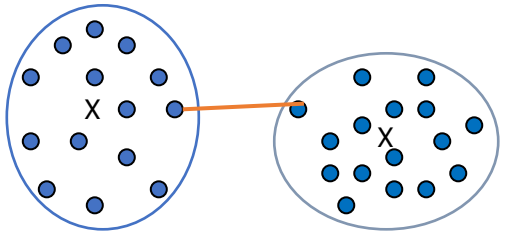
Agglomerative Clustering Algorithm

- AGNES (AGglomerative NESting) (Kaufmann and Rousseeuw, 1990)
  - Use the single-link method and the dissimilarity matrix
  - Continuously merge nodes that have the least dissimilarity
  - Eventually all nodes belong to the same cluster
- Agglomerative clustering varies on different similarity measures among clusters
  - Single link (nearest neighbor)
  - Complete link (diameter)
  - Average link (group average)
  - Centroid link (centroid similarity)

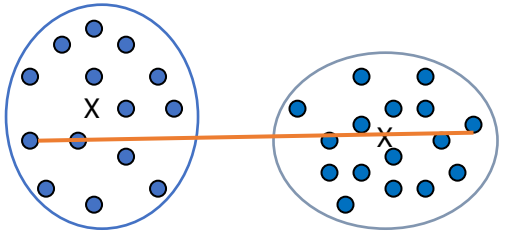
Agglomerative Clustering Algorithm



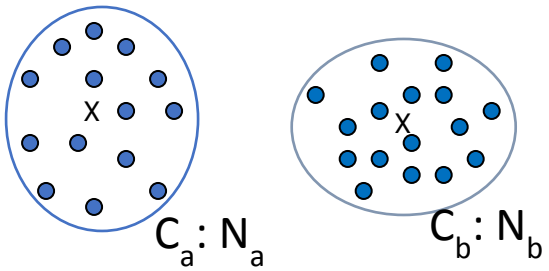
Single Link vs. Complete Link in Hierarchical Clustering



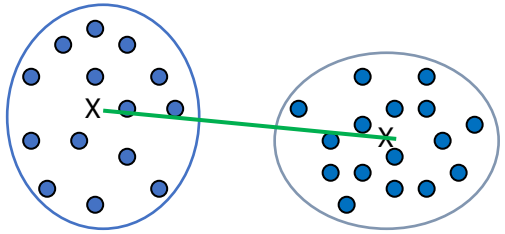
- Single link (nearest neighbor)
  - The similarity between two clusters is the similarity between their most similar (nearest neighbor) members
  - Local similarity-based: Emphasizing more on close regions, ignoring the overall structure of the cluster
  - Capable of clustering non-elliptical shaped group of objects
  - Sensitive to noise and outliers
- Complete link (diameter)
  - The similarity between two clusters is the similarity between their most dissimilar members
  - Merge two clusters to form one with the smallest diameter
  - Nonlocal in behavior, obtaining compact shaped clusters
  - Sensitive to outliers



Agglomerative Clustering: Average vs. Centroid Links



- Agglomerative clustering with average link
  - Average link: The average distance between an element in one cluster and an element in the other (i.e., all pairs in two clusters)
    - Expensive to compute



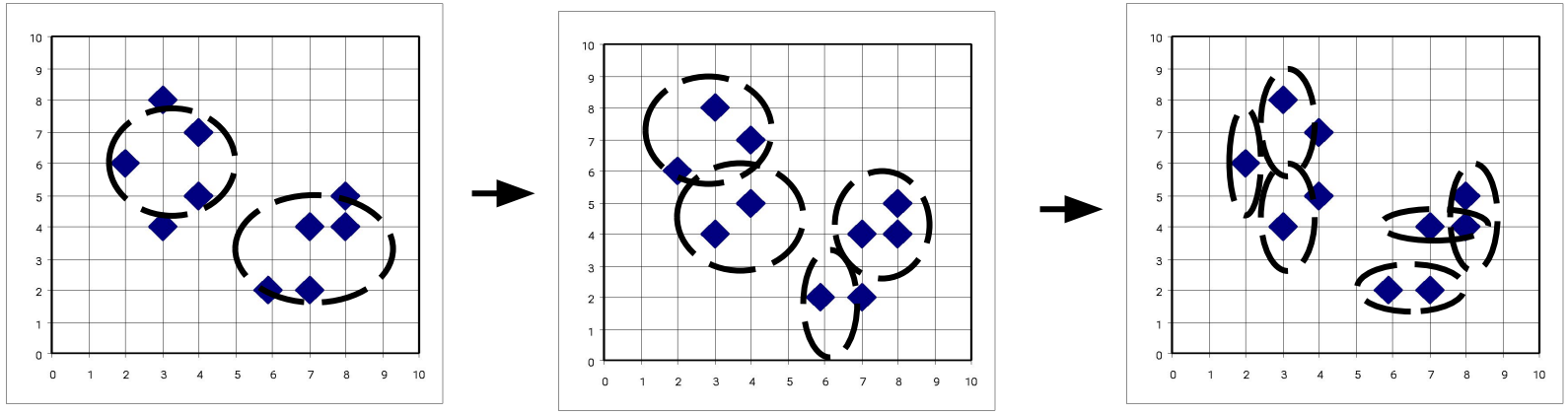
- Agglomerative clustering with centroid link
  - Centroid link: The distance between the centroids of two clusters
- Group Averaged Agglomerative Clustering (GAAC)
  - Let two clusters C_a and C_b be merged into $C_a \cup C_b$
  - The new centroid is $c_{a \cup b} = \frac{N_a c_a + N_b c_b}{N_a + N_b}$, where N_a and c_a are the cardinality and centroid of cluster C_a, respectively
  - The similarity measure for GAAC is the average of their distances

Agglomerative Clustering with Ward's Criterion

- Suppose two disjoint clusters C_i and C_j are merged, and m_{ij} is the mean of the new cluster
- Ward's criterion: $W\left(C_i, C_j\right) = \frac{n_i n_j}{n_i + n_j} \left|m_i - m_j\right|^2$

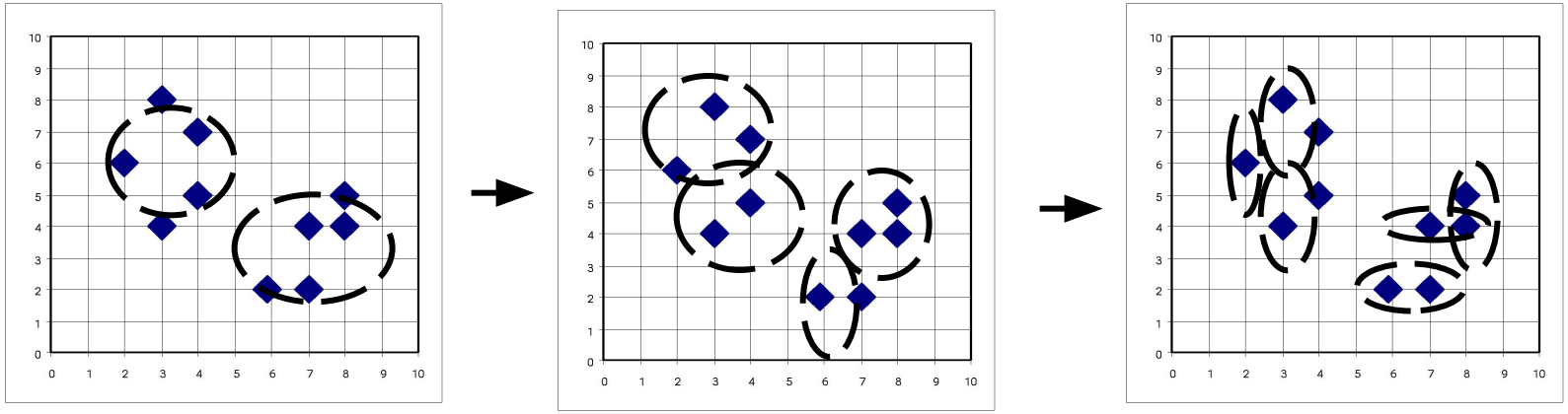
Divisive Clustering

- DIANA (Divisive Analysis) (Kaufmann and Rousseeuw,1990)
  - Implemented in some statistical analysis packages, e.g., Splus
- Inverse order of AGNES: Eventually each node forms a cluster on its own



Divisive Clustering Is a Top-down Approach

- The process starts at the root with all the points as one cluster
- It recursively splits the higher level clusters to build the dendrogram
- Can be considered as a global approach
- More efficient when compared with agglomerative clustering



More on Algorithm Design for Divisive Clustering

- Choosing which cluster to split
  - Check the sums of squared errors of the clusters and choose the one with the largest value
- Splitting criterion: Determining how to split
  - One may use Ward's criterion to chase for greater reduction in the difference in the SSE criterion as a result of a split
  - For categorical data, Gini-index can be used
- Handling the noise
  - Use a threshold to determine the termination criterion (do not generate clusters that are too small because they contain mainly noises)

Extensions to Hierarchical Clustering

- Weakness of the agglomerative & divisive hierarchical clustering methods
  - No revisit: cannot undo any merge/split decisions made before
  - Scalability bottleneck: Each merge/split needs to examine many possible options
    - Time complexity: at least $O(n^2)$, where n is the number of total objects
- Several other hierarchical clustering algorithms
  - BIRCH (1996): Use CF-tree and incrementally adjust the quality of sub-clusters
  - CURE (1998): Represent a cluster using a set of well-scattered representative points
  - CHAMELEON (1999): Use graph partitioning methods on the K-nearest neighbor graph of the data

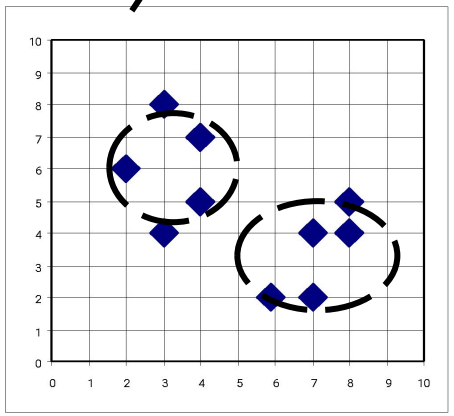
BIRCH: A Multi-Phase Hierarchical Clustering Method

- BIRCH (Balanced Iterative Reducing and Clustering Using Hierarchies)
  - Developed by Zhang, Ramakrishnan & Livny (SIGMOD'96)
  - Impact many new clustering methods and applications (received 2006 SIGMOD Test of Time award)
- Major innovation
  - Integrating hierarchical clustering (initial micro-clustering phase) and other clustering methods (at the later macro-clustering phase)
- Multi-phase hierarchical clustering
  - Phase1 (initial micro-clustering): Scan DB to build an initial CF tree, a multi-level compression of the data to preserve the inherent clustering structure of the data
  - Phase 2 (later macro-clustering): Use an arbitrary clustering algorithm (e.g., iterative partitioning) to cluster flexibly the leaf nodes of the CF-tree

Clustering Feature Vector

- Consider a cluster of multi-dimensional data objects/points
- The clustering feature (CF) of the cluster is a 3-D vector summarizing info about clusters of objects
  - Register the 0-th, 1st, and 2nd moments of a cluster
- Clustering Feature (CF): ***CF** = <n, **LS**, SS>*
  - *n*: Number of data points
  - *LS: linear sum of n points:* $\boldsymbol{LS} = \sum_{i=1}^{n} \boldsymbol{x}_i$
  - *SS: square sum of n points:* $SS = \sum_{i=1}^{n} \|\boldsymbol{x}_i\|^2$

CF_1 = <5, (16,30), 244>



(3,4)
(2,6)
(4,5)
(4,7)
(3,8)

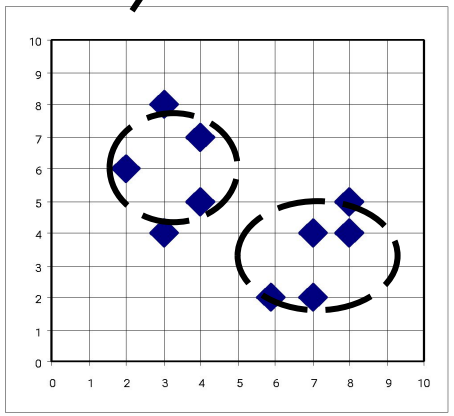
n = 5; **LS** = ((3+2+4+4+3), (4+6+5+7+8)) = (16, 30);
SS=$(3^2+2^2+4^2+4^2+3^2) + (4^2+6^2+5^2+7^2+8^2)$= 54+190=244

Clustering Feature: a Summary of the Statistics for the Given Cluster

- Registers crucial measurements for computing cluster and utilizes storage efficiently
- Clustering features are additive: Merging clusters C1 and C2—linear summation of CFs

$$CF_1 + CF_2 = \langle n_1 + n_2, LS_1 + LS_2, SS_1 + SS_2 \rangle$$

CF_1 = <5, (16,30), 244>



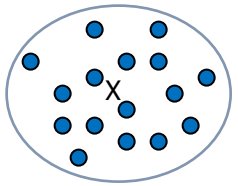
(3,4)
(2,6)
(4,5)
(4,7)
(3,8)

n = 5; **LS** = ((3+2+4+4+3), (4+6+5+7+8)) = (16, 30);
SS=(3^2+2^2+4^2+4^2+3^2) + (4^2+6^2+5^2+7^2+8^2)= 54+190=244

Essential Measures of Cluster: Centroid, Radius and Diameter

$$\boldsymbol{x}_0 = \frac{\sum_{i=1}^{n} \boldsymbol{x}_i}{n} = \frac{\boldsymbol{LS}}{n}$$

- Centroid: x_0
  - The "middle" of a cluster
  - n: number of points in a cluster
  - x_i is the i-th point in the cluster



$$R = \sqrt{\frac{\sum_{i=1}^{n} \|\boldsymbol{x}_i - \boldsymbol{x}_0\|^2}{n}} = \sqrt{\frac{SS}{n} - \frac{\|\boldsymbol{LS}\|^2}{n^2}}$$

$$D = \sqrt{\frac{\sum_{i=1}^{n} \sum_{j=1}^{n} \left\|\boldsymbol{x}_i - \boldsymbol{x}_j\right\|^2}{n(n-1)}} = \sqrt{\frac{2n \cdot SS - 2\|\boldsymbol{LS}\|^2}{n(n-1)}}$$

- Radius: R
  - Average distance from member objects to the centroid
  - The square root of average distance from any point of the cluster to its centroid
- Diameter: D
  - Average pairwise distance within a cluster
  - The square root of average mean squared distance between all pairs of points in the cluster

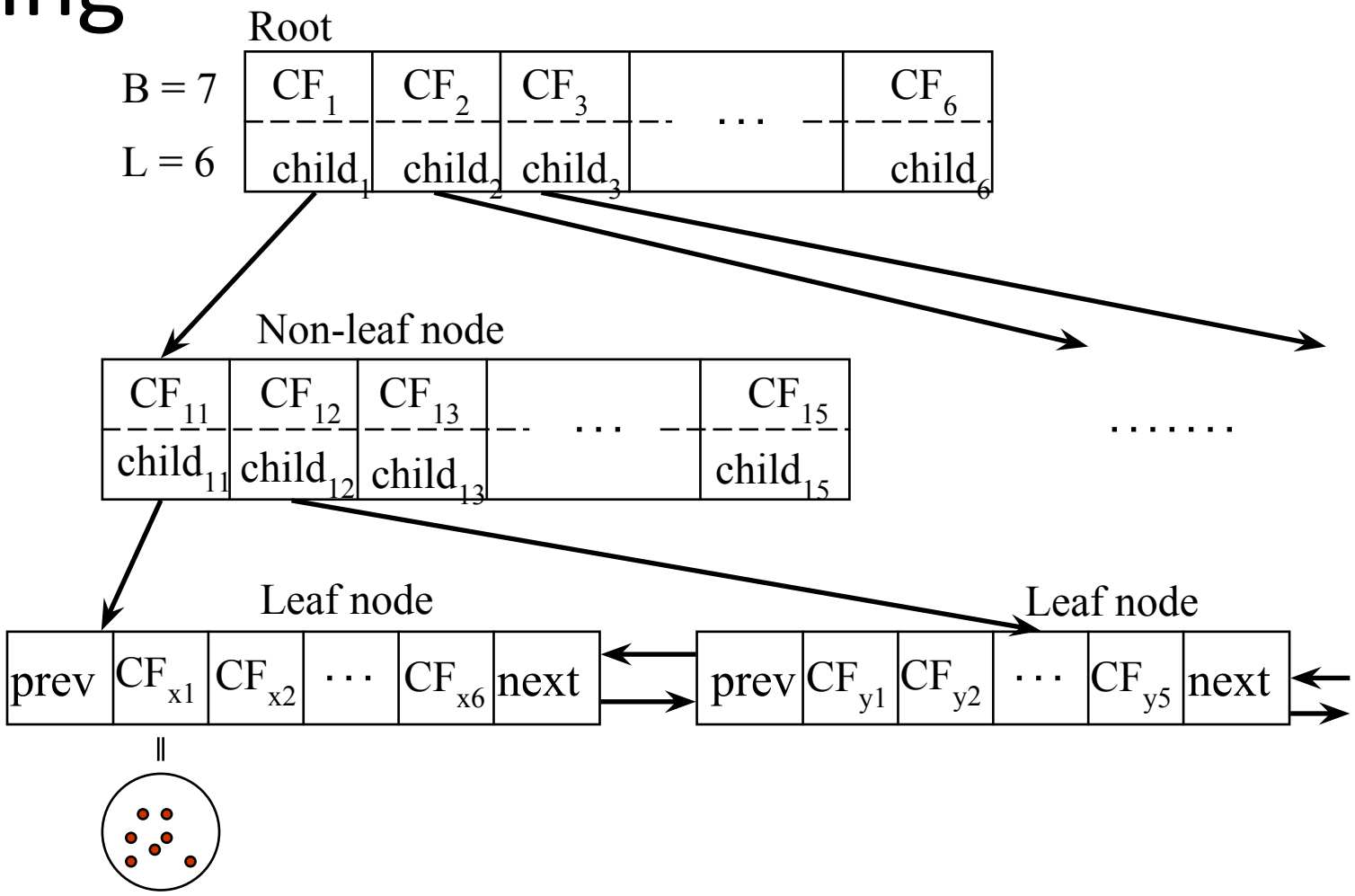
Example

- Continuing the previous example:
- $CF = < 5, (16, 30), 244 >$
- $\|\boldsymbol{LS}\|^2 = 16^2 + 30^2 = 1156$
- $R = \sqrt{244/5 - 1156/5^2} = 1.60$
- $D = \sqrt{(2\times5\times244 - 2\times1156)/(5\times4)} = 2.53$

CF Tree: A Height-Balanced Tree Storing Clustering Features for Hierarchical Clustering

- Incremental insertion of new points (similar to B+-tree)
- For each point in the input
  - Find its closest leaf entry
  - Add point to leaf entry and update CF
  - If entry diameter > max_diameter
    - Split leaf, and possibly parents
- A CF tree has two parameters
  - Branching factor: Maximum number of children
  - Maximum diameter of sub-clusters stored at the leaf nodes
- A CF tree: A height-balanced tree that stores the clustering features (CFs)
- The non-leaf nodes store sums of the CFs of their children

CF Tree: A Height-Balanced Tree Storing Clustering Features for Hierarchical Clustering

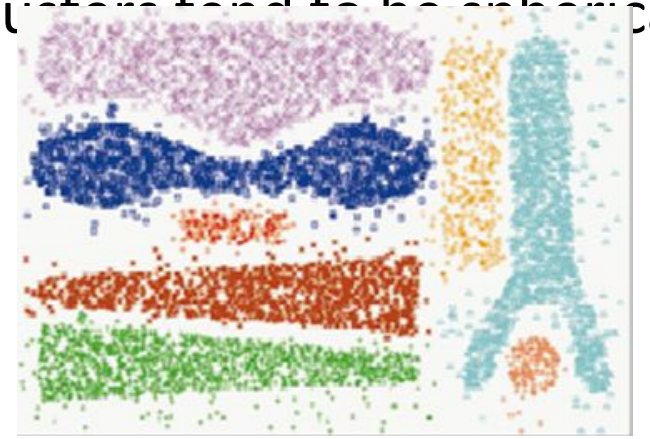


BIRCH: A Scalable and Flexible Clustering Method

- An integration of agglomerative clustering with other (flexible) clustering methods
- Low-level micro-clustering
  - Exploring CF-feature and BIRCH tree structure
  - Preserving the inherent clustering structure of the data
- Higher-level macro-clustering
  - Provide sufficient flexibility for integration with other clustering methods

BIRCH: Pros and Cons

- Strength: Good quality of clustering; linear scalability in large/stream databases; effective for incremental and dynamic clustering of incoming objects
- Weaknesses
  - Due to the fixed size of leaf nodes, clusters so formed may not be very natural
  - Clusters tend to be spherical given the radius and diameter measures



Images like this may give BIRCH a hard time

Probabilistic Hierarchical Clustering

- Algorithmic hierarchical clustering
  - Nontrivial to choose a good distance measure
  - Hard to handle missing attribute values
  - Optimization goal not clear: heuristic, local search
- Probabilistic hierarchical clustering
  - Use probabilistic models to measure distances between clusters
  - Generative model: Regard the set of data objects to be clustered as a sample of the underlying data generation mechanism to be analyzed
  - Easy to understand, same efficiency as algorithmic agglomerative clustering method, can handle partially observed data
- In practice, assume the generative models adopt common distribution functions, e.g., Gaussian distribution or Bernoulli distribution, governed by parameters

Generative Model

- Given a set of 1-D points X = {x1, …, xn} for clustering analysis & assuming they are generated by a Gaussian distribution $N(\mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$
- The probability that a point xi ∈ X is generated by the model: $P(x_i|\mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x_i-\mu)^2}{2\sigma^2}}$
- The likelihood that a data set X is generated by the model: $L(N(\mu, \sigma^2): X) = P(X|\mu, \sigma^2) = \prod_{i=1}^{n} \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x_i-\mu)^2}{2\sigma^2}}$
- The task of learning the generative model: find the parameters $N(\mu_0, \sigma_0^2) = \arg\max\{L(N(\mu, \sigma^2): X)\}$

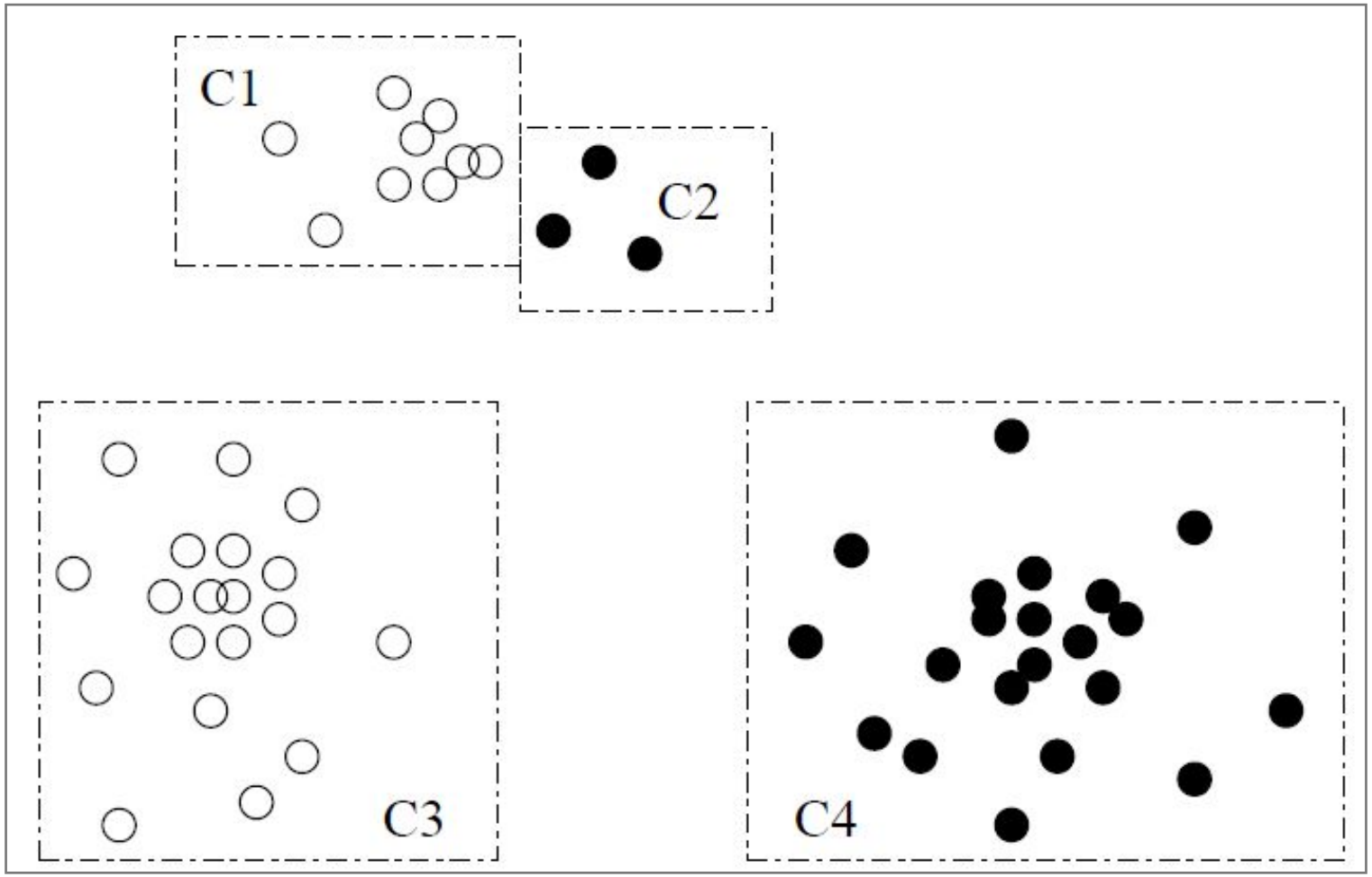
A Probabilistic Hierarchical Clustering Algorithm

- For a set of objects partitioned into m clusters C1, . . . ,Cm, the quality can be measured by $Q(\{C_1, \dots, C_m\}) = \prod_{i=1}^{m} P(C_i)$, where P() is the maximum likelihood
- If we merge two clusters C_{j1} and C_{j2} into a cluster $C_{j1} \cup C_{j2}$, the change in quality of the overall clustering is

$$Q\left(\left(\{C_1, \dots, C_m\} \setminus \{C_{j1}, C_{j2}\}\right) \cup \{C_{j1} \cup C_{j2}\}\right) - Q(\{C_1, \dots, C_m\})$$
$$= \prod_{i=1}^{m} P(C_i)\left(\frac{P(C_{j1} \cup C_{j2})}{P(C_{j1})P(C_{j2})} - 1\right)$$

- Distance between clusters C_i and C_j $dist(C_i, C_j) = -\log \frac{P(C_i \cup C_j)}{P(C_i)P(C_j)}$
- If $dist(C_i, C_j) < 0$, merge C_i and C_j

Example



Outline

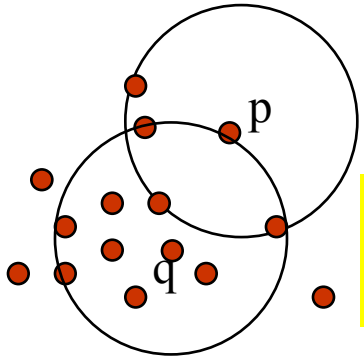
- Cluster analysis
- Partitioning methods
- Hierarchical methods
- Density-based and grid-based methods
  - DBSCAN: density-based clustering based on connected regions with high density
  - DENCLUE: clustering based on density distribution functions
  - Grid-based methods
- Evaluation of clustering

Density-Based Clustering Methods

- Clustering based on density (a local cluster criterion), such as density-connected points
- Major features:
  - Discover clusters of arbitrary shape
  - Handle noise
  - One scan (only examine the local region to justify density)
  - Need density parameters as termination condition
- Several interesting studies:
  - DBSCAN: Ester, et al. (KDD’96)
  - OPTICS: Ankerst, et al (SIGMOD’99)
  - DENCLUE: Hinneburg & D. Keim (KDD’98)
  - CLIQUE: Agrawal, et al. (SIGMOD’98) (also, grid-based)

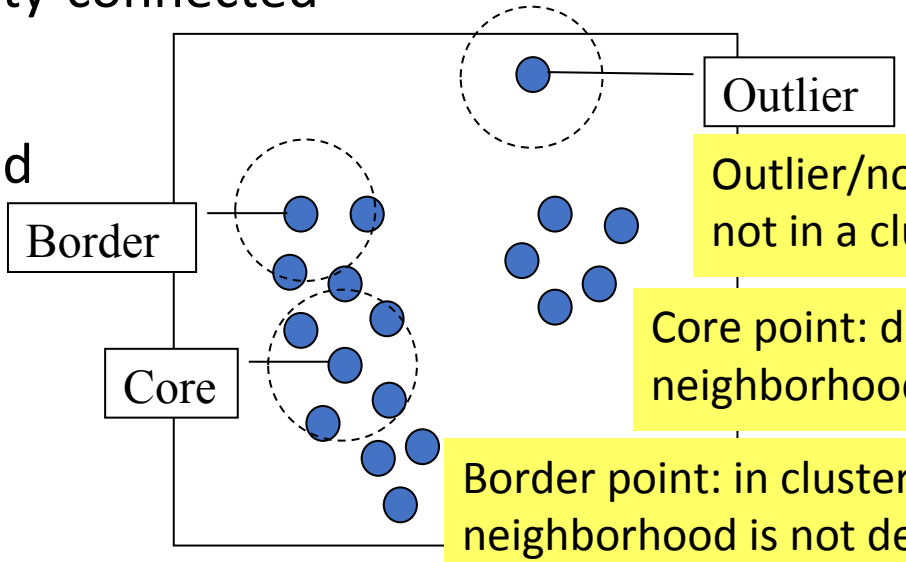
DBSCAN: A Density-Based Spatial Clustering Algorithm

- DBSCAN (M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, KDD'96)
  - Discovers clusters of arbitrary shape: Density-Based Spatial Clustering of Applications with Noise
- A density-based notion of cluster
  - A cluster is defined as a maximal set of density-connected points
  - Two parameters:
  - Eps (ε): Maximum radius of the neighborhood
  - MinPts: Minimum number of points in the
    - Eps-neighborhood of a point
- The Eps(ε)-neighborhood of a point q:
  - NEps(q): {p belongs to D | dist(p, q) ≤ Eps}



MinPts = 5

Eps = 1 cm



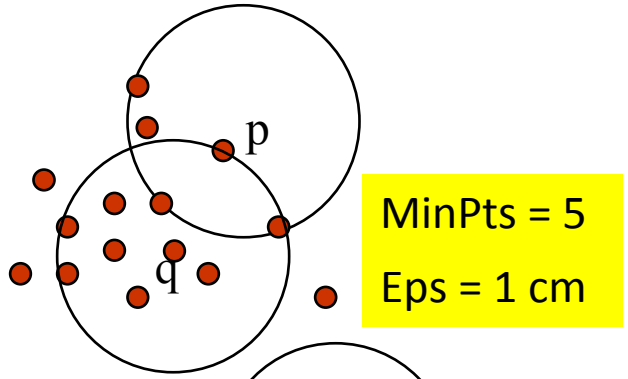
Outlier/noise: not in a cluster

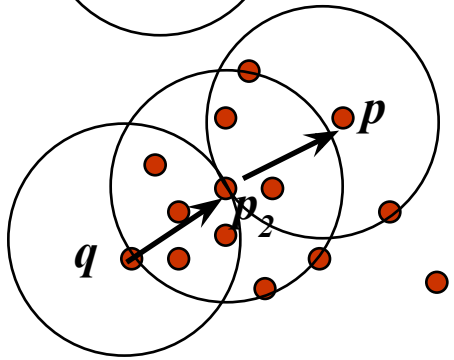
Core point: dense neighborhood

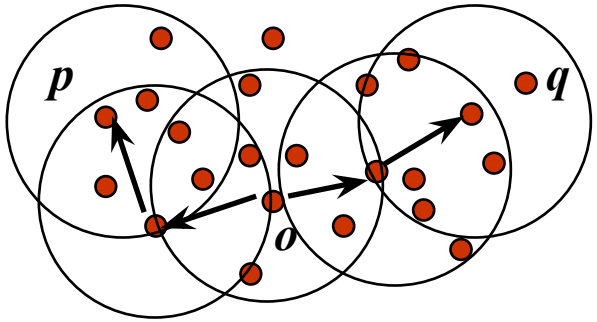
Border point: in cluster but neighborhood is not dense

DBSCAN: Density-Reachable and Density-Connected

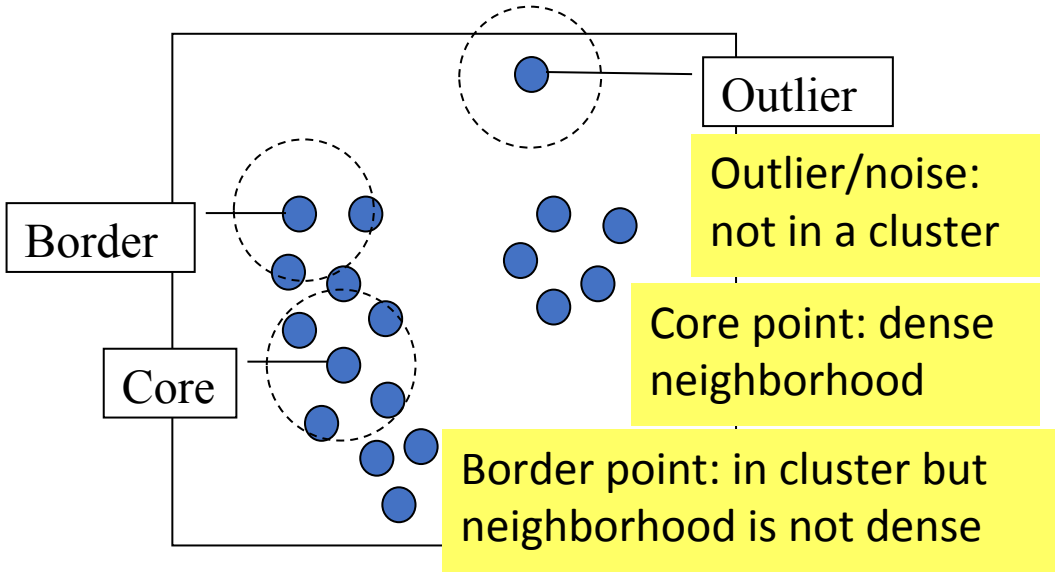
- Directly density-reachable:
  - A point p is directly density-reachable from a point q w.r.t. Eps (ε), MinPts if
    - p belongs to NEps(q)
    - core point condition: |NEps (q)| ≥ MinPts
- Density-reachable:
  - A point p is density-reachable from a point q w.r.t. Eps, MinPts if there is a chain of points $p_1, \ldots, p_n, p_1 = q, p_n = p$ such that $p_i + 1$ is directly density-reachable from p_i
- Density-connected:
  - A point p is density-connected to a point q w.r.t. Eps, MinPts if there is a point o such that both p and q are density-reachable from o w.r.t. Eps and MinPts







DBSCAN: The Algorithm



- Algorithm
  - Arbitrarily select a point p
  - Retrieve all points density-reachable
    - from p w.r.t. Eps and MinPts
  - If p is a core point, a cluster is formed
  - If p is a border point, no points are directly density-reachable from p, and DBSCAN visits the next point of the database
  - Continue the process until all of the points have been processed
- Computational complexity
  - If a spatial index is used, the computational complexity of DBSCAN is $O(n \log n)$, where n is the number of database objects
  - Otherwise, the complexity is $O(n^2)$

DBSCAN Is Sensitive to the Setting of Parameters

Figure 8. DBScan results for DS1 with MinPts at 4 and Eps at (a) 0.5 and (b) 0.4.

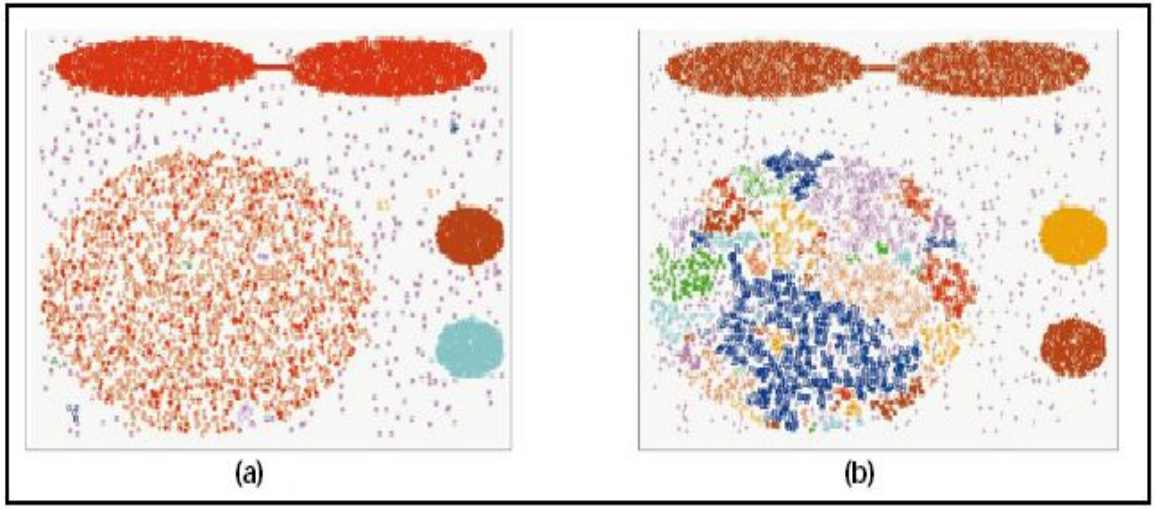
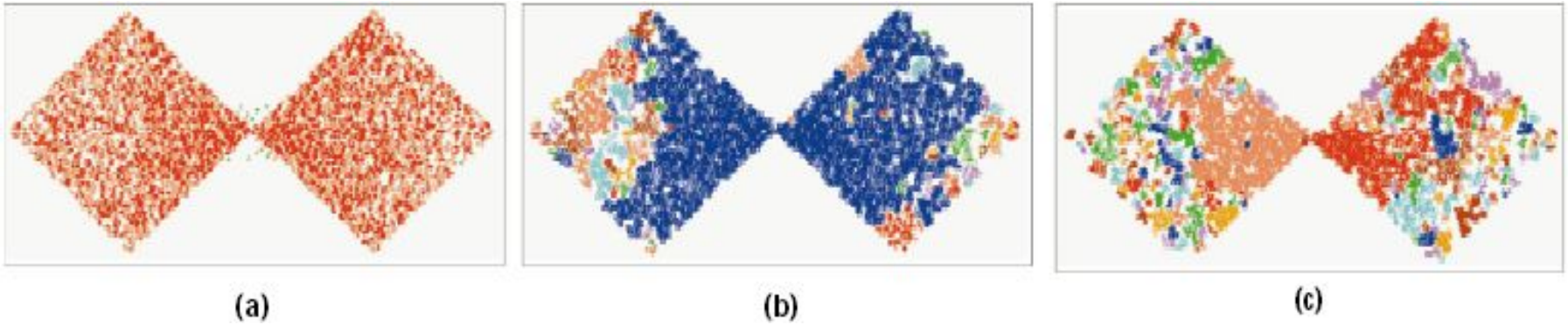


Figure 9. DBScan results for DS2 with MinPts at 4 and Eps at (a) 5.0, (b) 3.5, and (c) 3.0.



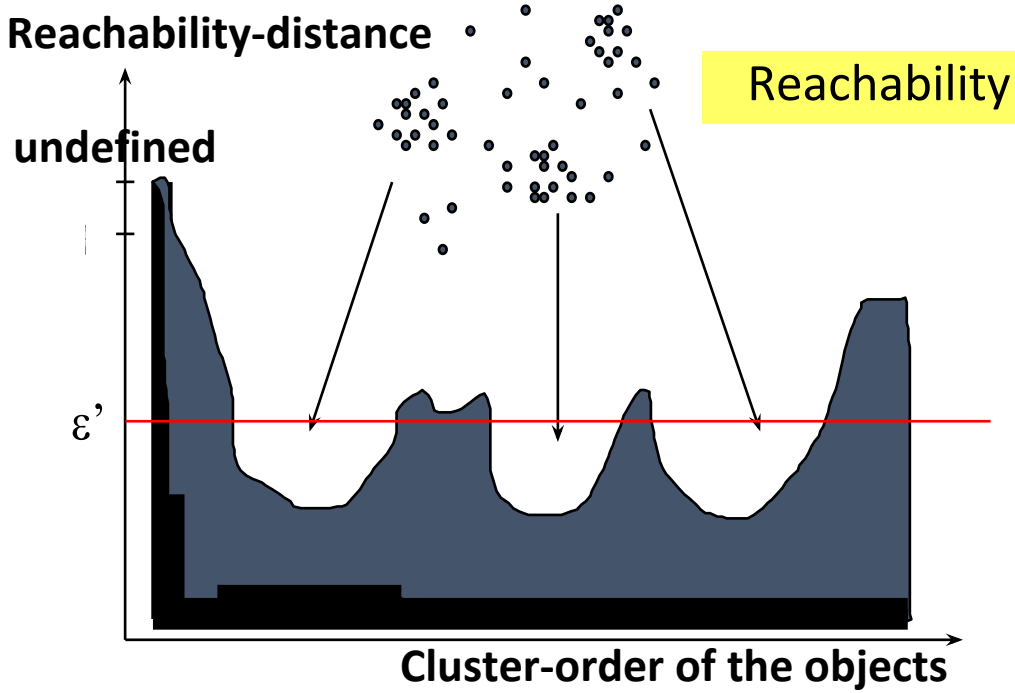
Ack. Figures from G. Karypis, E.-H. Han, and V. Kumar, *COMPUTER*, 32(8), 1999

OPTICS: Ordering Points To Identify Clustering Structure

- OPTICS (Ankerst, Breunig, Kriegel, and Sander, SIGMOD'99)
  - DBSCAN is sensitive to parameter setting
  - An extension: finding clustering structure
- Observation: Given a MinPts, density-based clusters w.r.t. a higher density are completely contained in clusters w.r.t. to a lower density
- Idea: Higher density points should be processed first—find high-density clusters first
- OPTICS stores such a clustering order using two pieces of information:
  - Core distance and reachability distance

Visualization

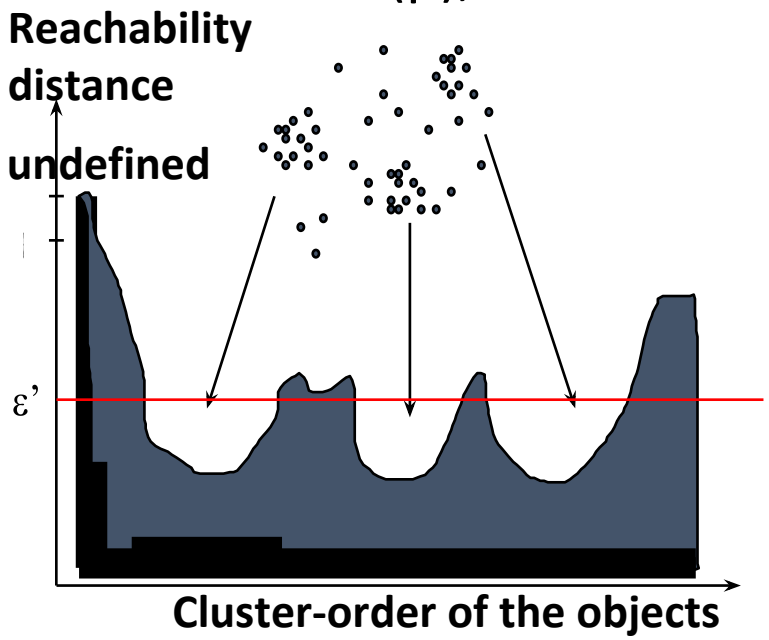
- Since points belonging to a cluster have a low reachability distance to their nearest neighbor, valleys correspond to clusters
- The deeper the valley, the denser the cluster



Reachability plot for a dataset

OPTICS: An Extension from DBSCAN

- Core distance of an object p: The smallest value ε such
  - that the ε-neighborhood of p has at least MinPts objects
  - Let Nε(p): ε-neighborhood of p, where ε is a distance value
  - Core-distanceε, MinPts(p) = Undefined if card(Nε(p)) < MinPts; MinPts-distance(p), otherwise



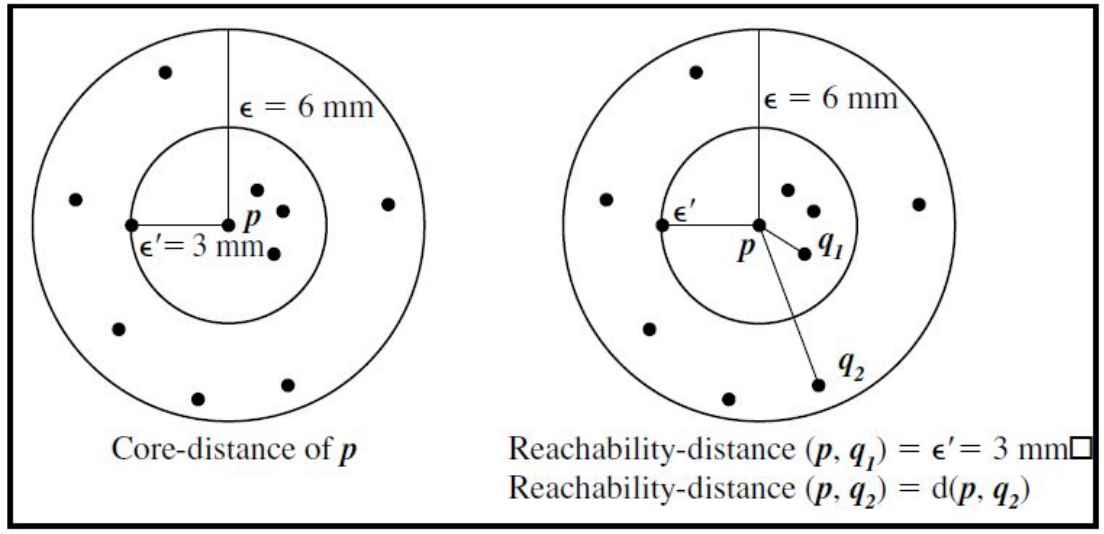


Figure 10.16: OPTICS terminology. Based on [ABKS99].

OPTICS: An Extension from DBSCAN

- Reachability distance of object *q* from core object *p* is the min. radius value that makes *q* density-reachable from *p*

  Reachability-distance$_{\varepsilon, MinPts}$(*p*, *q*) =

  Undefined, *if p is not a core object*

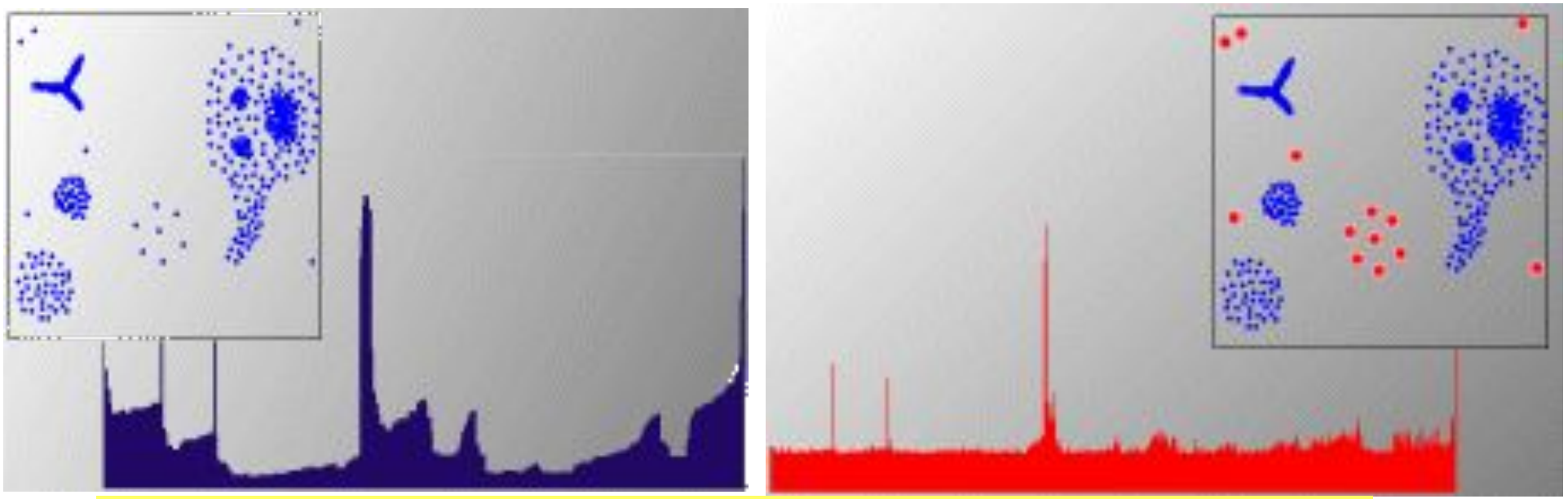
  max(core-distance(*p*), distance (*p*, *q*)), *otherwise*

- Complexity: O(*N* log*N*) (if index-based), where *N*: # of points

OPTICS: Finding Hierarchically Nested Clustering Structures

- OPTICS produces a special cluster-ordering of the data points with respect to its density-based clustering structure
- The cluster-ordering contains information equivalent to the density-based clusterings corresponding to a broad range of parameter settings
- Good for both automatic and interactive cluster analysis—finding intrinsic, even hierarchically nested clustering structures

OPTICS: Finding Hierarchically Nested Clustering Structures



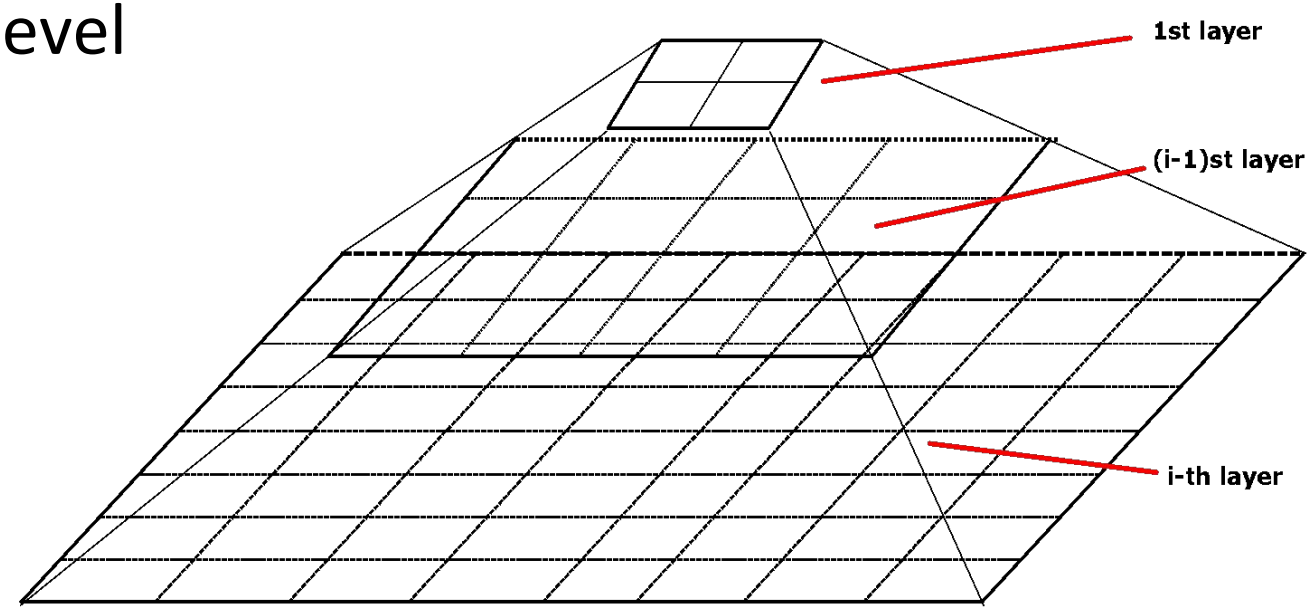
Finding nested clustering structures with different parameter settings

Grid-Based Clustering Methods

- Grid-Based Clustering: Explore multi-resolution grid data structure in clustering
  - Partition the data space into a finite number of cells to form a grid structure
  - Find clusters (dense regions) from the cells in the grid structure
- Features and challenges of a typical grid-based algorithm
  - Efficiency and scalability: # of cells << # of data points
  - Uniformity: Uniform, hard to handle highly irregular data distributions
  - Locality: Limited by predefined cell sizes, borders, and the density threshold
  - Curse of dimensionality: Hard to cluster high-dimensional data
- Methods to be introduced
  - STING (a STatistical INformation Grid approach) (Wang, Yang and Muntz, VLDB'97)
  - CLIQUE (Agrawal, Gehrke, Gunopulos, and Raghavan, SIGMOD'98)
    - Both grid-based and subspace clustering

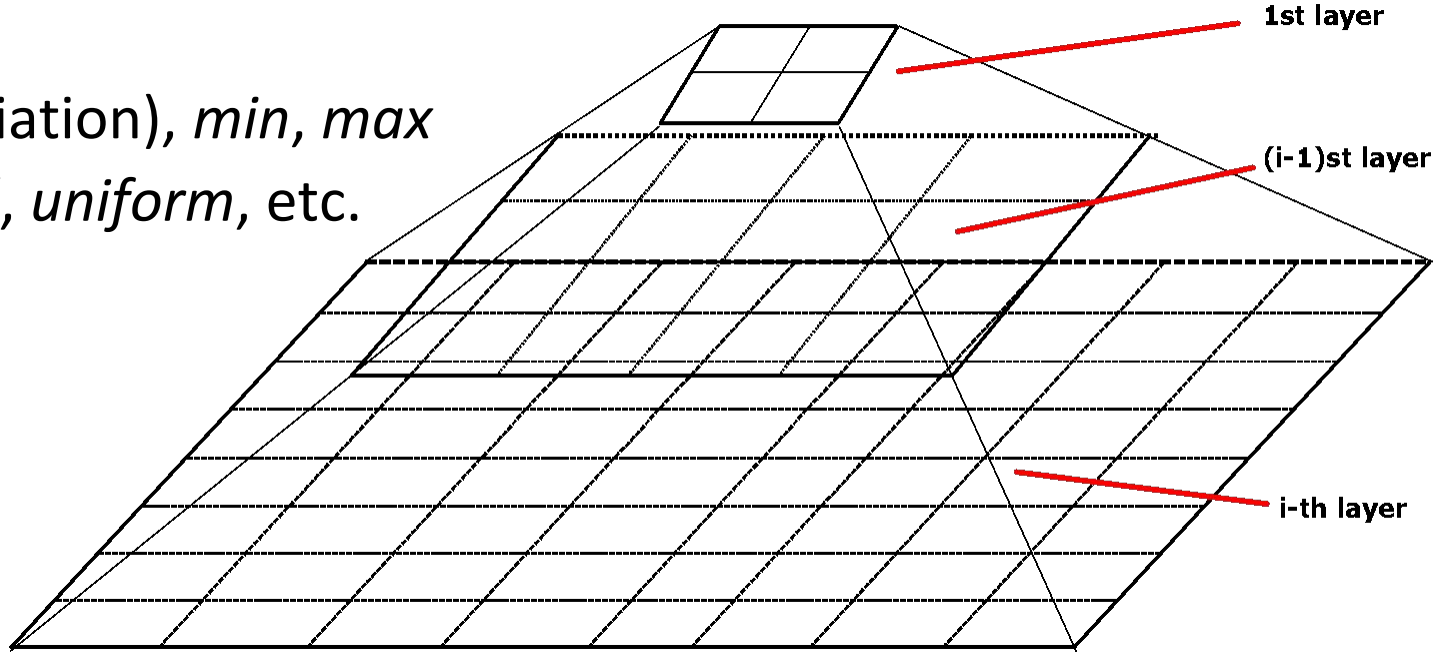
STING: A Statistical Information Grid Approach

- STING (Statistical Information Grid) (Wang, Yang and Muntz, VLDB'97)
- The spatial area is divided into rectangular cells at different levels of resolution, and these cells form a tree structure
- A cell at a high level contains a number of smaller cells of the next lower level



STING: A Statistical Information Grid Approach

- Statistical information of each cell is calculated and stored beforehand and is used to answer queries
- Parameters of higher level cells can be easily calculated from that of lower level cell, including
  - *count*, *mean*, *s*(standard deviation), *min*, *max*
  - type of distribution—*normal*, *uniform*, etc.



Query Processing in STING and Its Analysis

- To process a region query
  - Start at the root and proceed to the next lower level, using the STING index
  - Calculate the likelihood that a cell is relevant to the query at some confidence level using the statistical information of the cell
  - Only children of likely relevant cells are recursively explored
  - Repeat this process until the bottom layer is reached
- Advantages
  - Query-independent, easy to parallelize, incremental update
  - Efficiency: Complexity is O(K)
    - K: # of grid cells at the lowest level, and K << N (i.e., # of data points)
- Disadvantages
  - Its probabilistic nature may imply a loss of accuracy in query processing

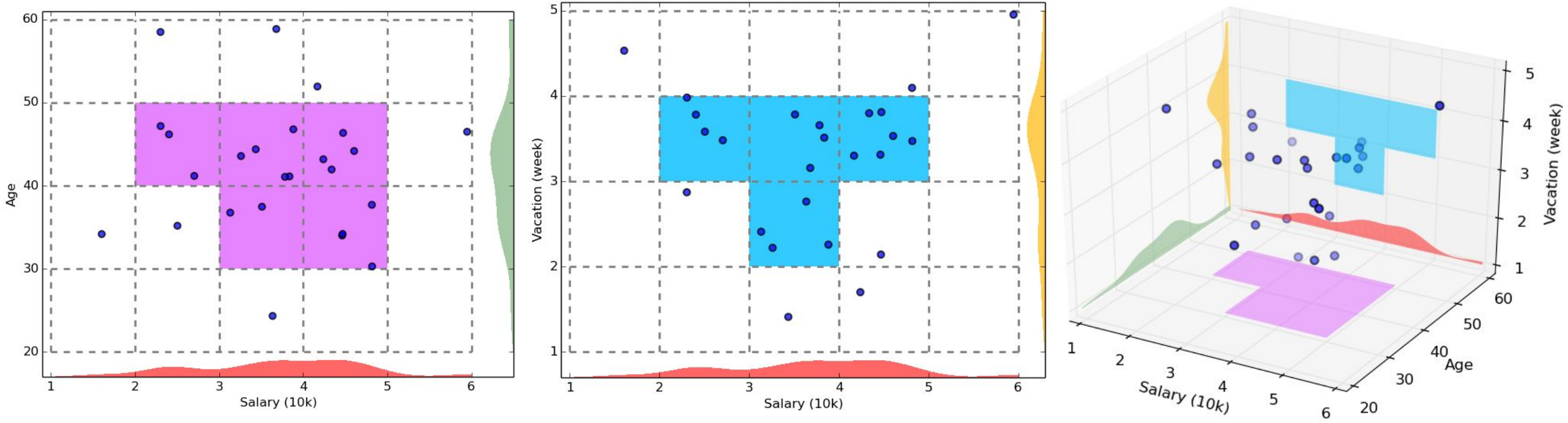
CLIQUE: Grid-Based Subspace Clustering

- CLIQUE (Clustering In QUEst) (Agrawal, Gehrke, Gunopulos, Raghavan: SIGMOD'98)
- CLIQUE is a density-based and grid-based subspace clustering algorithm
  - Grid-based: It discretizes the data space through a grid and estimates the density by counting the number of points in a grid cell
  - Density-based: A cluster is a maximal set of connected dense units in a subspace
    - A unit is dense if the fraction of total data points contained in the unit exceeds the input model parameter
  - Subspace clustering: A subspace cluster is a set of neighboring dense cells in an arbitrary subspace. It also discovers some minimal descriptions of the clusters
- It automatically identifies subspaces of a high dimensional data space that allow better clustering than original space using the Apriori principle

CLIQUE: SubSpace Clustering with Aprori Pruning

- Start at 1-D space and discretize numerical intervals in each axis into grid
- Find dense regions (clusters) in each subspace and generate their minimal descriptions
  - Use the dense regions to find promising candidates in 2-D space based on the Apriori principle
  - Repeat the above in level-wise manner in higher dimensional subspaces

CLIQUE: SubSpace Clustering with Aprori Pruning



Major Steps of the CLIQUE Algorithm

- Identify subspaces that contain clusters
  - Partition the data space and find the number of points that lie inside each cell of the partition
  - Identify the subspaces that contain clusters using the Apriori principle
- Identify clusters
  - Determine dense units in all subspaces of interests
  - Determine connected dense units in all subspaces of interests
- Generate minimal descriptions for the clusters
  - Determine maximal regions that cover a cluster of connected dense units for each cluster
  - Determine minimal cover for each cluster

Pros and Cons of CLIQUE

- Strengths
  - Automatically finds subspaces of the highest dimensionality as long as high density clusters exist in those subspaces
  - Insensitive to the order of records in input and does not presume some canonical data distribution
  - Scales linearly with the size of input and has good scalability as the number of dimensions in the data increases
- Weaknesses
  - As in all grid-based clustering approaches, the quality of the results crucially depends on the appropriate choice of the number and width of the partitions and grid cells

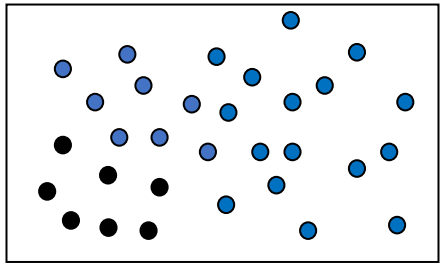
Outline

- Cluster analysis
- Partitioning methods
- Hierarchical methods
- Density-based and grid-based methods
- Evaluation of clustering
  - Assessing clustering tendency
  - Determining the number of clusters
  - Measuring clustering quality: extrinsic methods
  - Intrinsic methods

Evaluation of Clustering: Basic Concepts

- Evaluation of clustering
  - Assess the feasibility of clustering analysis on a data set
  - Evaluate the quality of the results generated by a clustering method
- Major issues on clustering assessment and validation
  - Clustering tendency: assessing the suitability of clustering: whether the data has any inherent grouping structure
  - Determining the Number of Clusters: determining for a dataset the right number of clusters that may lead to a good quality clustering
  - Clustering quality evaluation: evaluating the quality of the clustering results

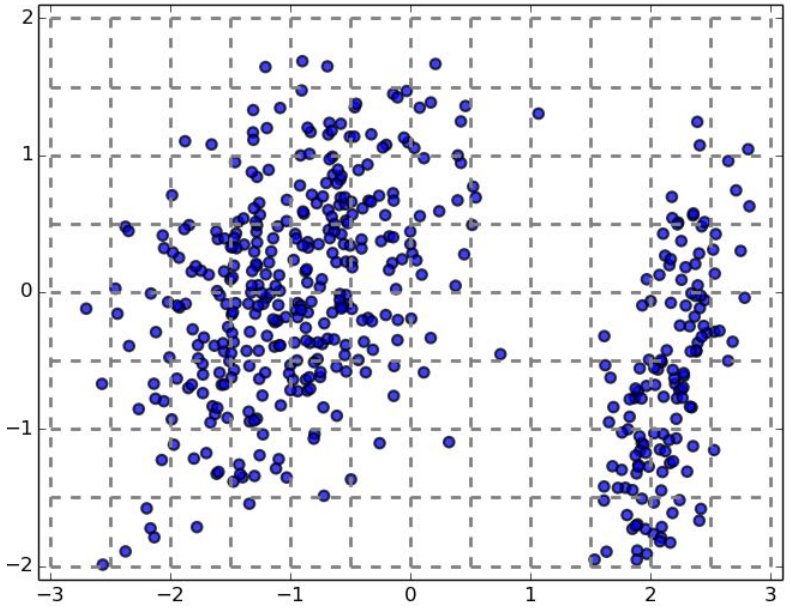
Clustering Tendency: Whether the Data Contains Inherent Grouping Structure



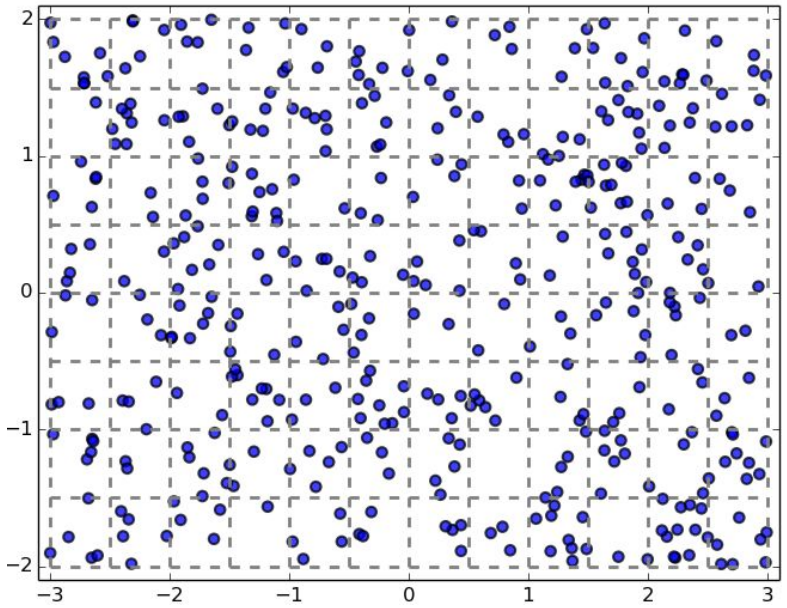
- Assess the suitability of clustering
  - Whether the data has any "inherent grouping structure" — non-random structure that may lead to meaningful clusters
- Determine clustering tendency or clusterability
  - A hard task because there are so many different definitions of clusters
    - Different definitions: Partitioning, hierarchical, density-based and graph-based
  - Even fixing a type, still hard to define an appropriate null model for a data set
- There are some clusterability assessment methods, such as
  - Spatial histogram: Contrast the histogram of the data with that generated from random samples
  - Distance distribution: Compare the pairwise point distance from the data with those from the randomly generated samples
  - Hopkins Statistic: A sparse sampling test for spatial randomness

Testing Clustering Tendency: A Spatial Histogram Approach

- Spatial Histogram Approach: Contrast the d-dimensional histogram of the input dataset D with the histogram generated from random samples
  - Dataset D is clusterable if the distributions of two histograms are rather different



(a) Input dataset



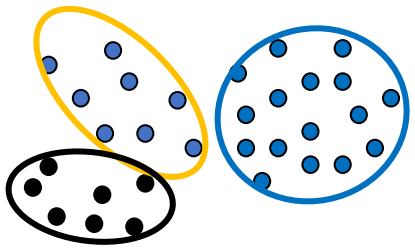
(b) Data generated from random samples

Testing Clustering Tendency: A Spatial Histogram Approach

- Method outline
  - Divide each dimension into equi-width bins, count how many points lie in each cell, and obtain the empirical joint probability mass function (EPMF)
- Do the same for the randomly sampled data
- Compute how much they differ using the Kullback-Leibler (KL) divergence value

Determining the Number of Clusters

- The appropriate number of clusters controls the proper granularity of cluster analysis
  - Finding a good balance between compressibility and accuracy in cluster analysis
- Two undesirable extremes
  - The whole data set is one cluster: No value of clustering
  - Treating each point as a cluster: No data summarization

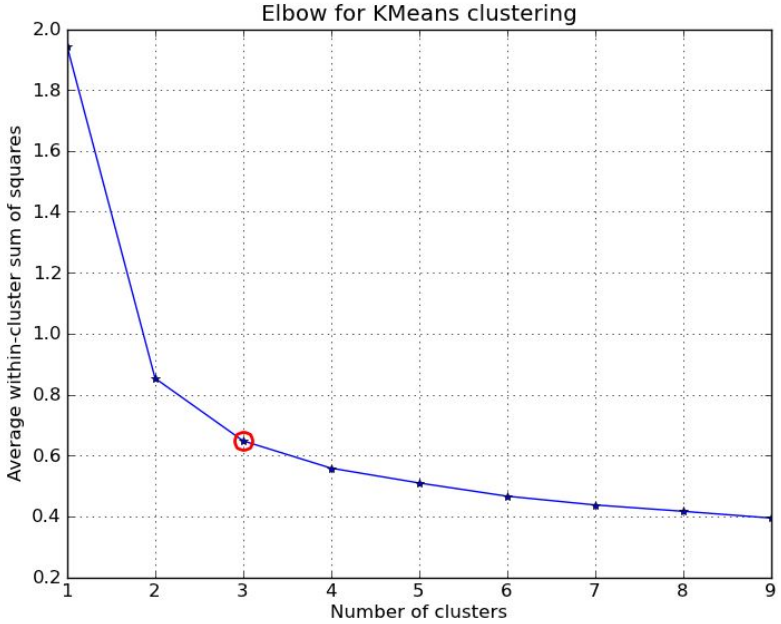


Determining the Number of Clusters

- The right number of clusters often depends on the distribution's shape and scale in the data set, as well as the clustering resolution required by the user
- Methods for determining the number of clusters
- An empirical method
  - # of clusters: $k \approx \sqrt{\frac{n}{2}}$ for a dataset of n points (e.g., n = 200, k = 10)
  - Each cluster is expected to have about $\sqrt{2n}$ points

Finding the Number of Clusters: the Elbow Method

- Use the turning point in the curve of the sum of within cluster variance with respect to the # of clusters
  - Increasing the # of clusters can help reduce the sum of within-cluster variance of each cluster
  - But splitting a cohesive cluster gives only a small reduction



Finding K, the Number of Clusters: A Cross Validation Method

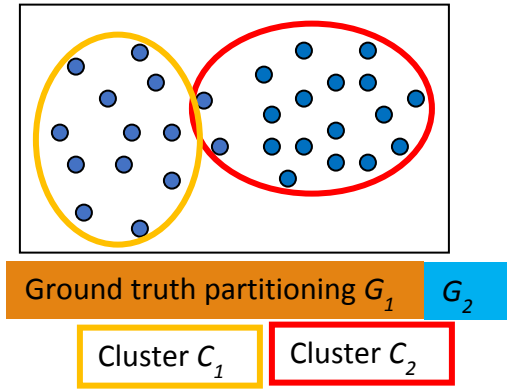
- Divide a given data set into m parts, and use m – 1 parts to obtain a clustering model
- Use the remaining part to test the quality of the clustering
  - For example, for each point in the test set, find the closest centroid, and use the sum of squared distance between all points in the test set and their closest centroids to measure how well the model fits the test set
- For any k > 0, repeat it m times, compare the overall quality measure w.r.t. different k's, and find # of clusters that fits the data the best

Measuring Clustering Quality

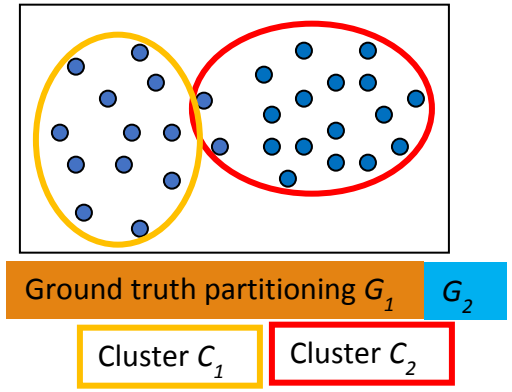
- Clustering Evaluation: Evaluating how good the clustering results are
  - No commonly recognized best suitable measure in practice
- Extrinsic vs. intrinsic methods: depending on whether ground truth is used
  - Ground truth: the ideal clustering built by using human experts
- Extrinsic: Supervised, employ criteria not inherent to the dataset
  - Compare a clustering against prior or expert-specified knowledge (i.e., the ground truth) using certain clustering quality measure
- Intrinsic: Unsupervised, criteria derived from data itself
  - Evaluate the goodness of a clustering by considering how well the clusters are separated and how compact the clusters are (e.g., silhouette coefficient)

General Criteria for Measuring Clustering Quality with Extrinsic Methods

- Given the ground truth Cg, Q(C, Cg) is the quality measure for a clustering C
- Q(C, Cg) is good if it satisfies the following four essential criteria
  - Cluster homogeneity: the purer, the better
  - Cluster completeness: assign objects belonging to the same category in the ground truth to the same cluster
  - Rag bag better than alien: putting a heterogeneous object into a pure cluster should be penalized more than putting it into a rag bag (i.e., “miscellaneous” or “other” category)
  - Small cluster preservation: splitting a small category into pieces is more harmful than splitting a large category into pieces

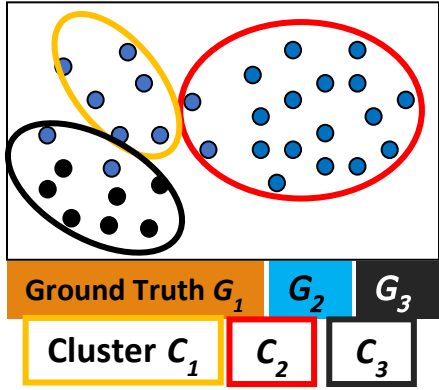


Commonly Used Extrinsic Methods



- Matching-based methods
  - Examine how well the clustering results match the ground truth in partitioning the objects in the data set
- Information theory-based methods
  - Compare the distribution of the clustering results and that of the ground truth
  - Information theory (e.g., entropy) used to quantify the comparison
  - Ex. Conditional entropy, normalized mutual information (NMI)
- Pairwise comparison-based methods
  - Treat each group in the ground truth as a class, and then check the pairwise consistency of the objects in the clustering results
  - Ex. Four possibilities: TP, FN, FP, TN; Jaccard coefficient

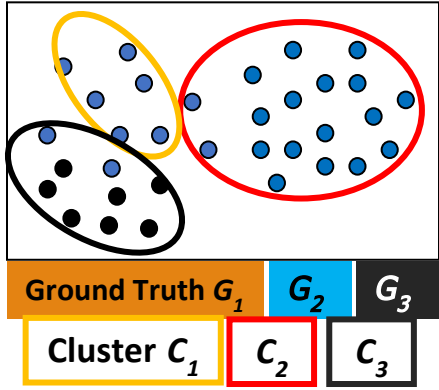
Matching-Based Methods



- The matching based methods compare clusters in the clustering results and the groups in the ground truth
- Suppose a clustering method partitions D = {o_1, …, o_n} into *m* clusters C = {C_1, …, C_m}. The ground truth G partitions D into *l* groups G = {G_1, …, G_l}
- Purity: the extent that cluster C_i contains points only from one (ground truth) partition
  - Purity for cluster C_i: $\frac{|C_i \cap G_j|}{|C_i|}$, where G_j matching G_j maximizes $|C_i \cap G_j|$
  - Total purity of clustering C:

$$purity = \sum_{i=1}^{m} \frac{|C_i|}{n} \max_{j=1}^{l} \left\{ \frac{|C_i \cap G_j|}{|C_i|} \right\} = \frac{1}{n} \sum_{i=1}^{m} \max_{j=1}^{l} \{ |C_i \cap G_j| \}$$

Matching-Based Methods: Example



- Consider 11 objects

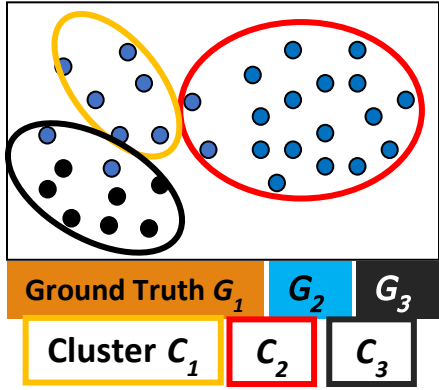
Object	a	b	c	d	e	f	g	h	i	j	k
Ground truth $\mathcal{G}$	G_1	G_1	G_1	G_1	G_1	G_1	G_2	G_2	G_2	G_2	G_3
Clustering $\mathcal{C}_1$	C_1	C_1	C_1	C_1	C_2	C_2	C_3	C_3	C_3	C_3	C_4
Clustering $\mathcal{C}_2$	C_1	C_1	C_2	C_2	C_2	C_3	C_1	C_2	C_2	C_1	C_3

Purity for clustering $\boldsymbol{C_1}$ = 1/11 $(4+2+4+1) = 11/11 = 1;$

Purity for clustering $\boldsymbol{C_2}$ = 1/11 $(2+3+1) = 6/11$

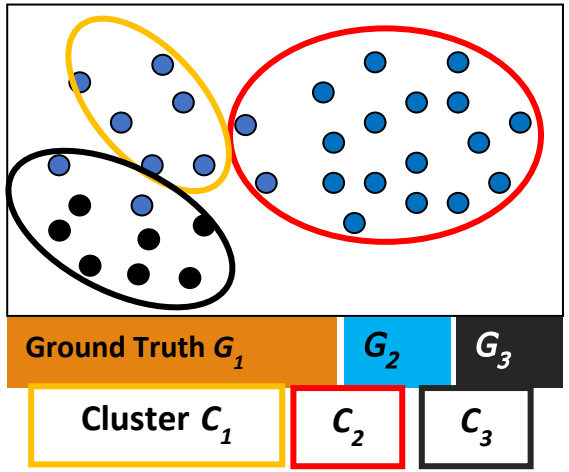
- Other methods:
  - maximum matching; F-measure

Information Theory-Based Methods (I) Conditional Entropy



- A clustering can be regarded as a compressed representation of a given set of objects
- The better the clustering results approach the ground-truth, the less amount of information is needed
- This idea leads to the use of *conditional entropy*

Information Theory-Based Methods (I) Conditional Entropy



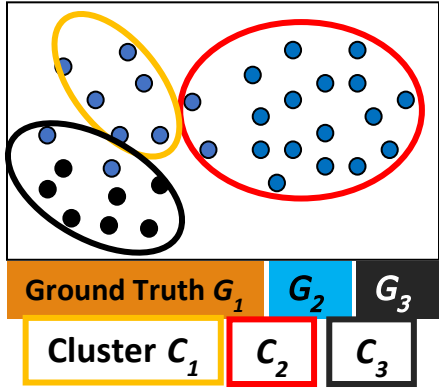
- Entropy of clustering C: $H(C) = -\sum_{i=1}^{m} \frac{|C_i|}{n} \log \frac{|C_i|}{n}$
- Entropy of ground truth G: $H(G) = -\sum_{i=1}^{l} \frac{|G_i|}{n} \log \frac{|G_i|}{n}$
- Conditional entropy of G given cluster C_i:

$$H(G|C_i) = -\sum_{j=1}^{l} \frac{|C_i \cap G_j|}{|C_i|} \log \frac{|C_i \cap G_j|}{|C_i|}$$

- Conditional entropy of G given clustering C:

$$H(G|C) = \sum_{i=1}^{m} \frac{|C_i|}{n} H(G|C_i) = -\sum_{i=1}^{m} \sum_{j=1}^{l} \frac{|C_i \cap G_j|}{n} \log \frac{|C_i \cap G_j|}{|C_i|}$$

Example



- Consider 11 objects

Object	a	b	c	d	e	f	g	h	i	j	k
Ground truth $\mathcal{G}$	G_1	G_1	G_1	G_1	G_1	G_1	G_2	G_2	G_2	G_2	G_3
Clustering $\mathcal{C}_1$	C_1	C_1	C_1	C_1	C_2	C_2	C_3	C_3	C_3	C_3	C_4
Clustering $\mathcal{C}_2$	C_1	C_1	C_2	C_2	C_2	C_3	C_1	C_2	C_2	C_1	C_3

Purity for clustering $\boldsymbol{C_1}$ = 1/11 $(4+2+4+1) = 11/11 = 1$;

Purity for clustering $\boldsymbol{C_2}$ = 1/11 $(2+3+1) = 6/11$

$$H(\mathcal{G}|\mathcal{C}_1) = -(\frac{4}{11}\log\frac{4}{4} + \frac{2}{11}\log\frac{2}{2} + \frac{4}{11}\log\frac{4}{4} + \frac{1}{11}\log\frac{1}{1}) = 0$$

$$H(\mathcal{G}|\mathcal{C}_2) = -(\frac{2}{11}\log\frac{2}{4} + \frac{2}{11}\log\frac{2}{4} + \frac{3}{11}\log\frac{3}{5} + \frac{2}{11}\log\frac{2}{5} + \frac{1}{11}\log\frac{1}{2} + \frac{1}{11}\log\frac{1}{2})$$
$$= 0.297$$

Note: conditional entropy cannot detect the issue that $\boldsymbol{C_1}$ splits the objects in G into two clusters

Information Theory-Based Methods (II) Normalized Mutual Information (NMI)

- Mutual information $I(C,G) = -\sum_{i=1}^{r}\sum_{j=1}^{k} p_{ij} \log \frac{p_{ij}}{p_{C_i} p_{G_j}}$
  - Quantify the amount of shared info between the clustering C and the ground-truth partitioning G
  - Measure the dependency between the observed joint probability p_{ij} of C and G, and the expected joint probability $p_{C_i} p_{G_j}$ under the independence assumption
  - When C and G are independent, $p_{ij} = p_{C_i} p_{G_j}$, I(C, G) = 0
  - However, there is no upper bound on the mutual information
- Normalized mutual information $NMI(C,G) = \sqrt{\frac{I(C,G)}{H(C)} \frac{I(C,G)}{H(G)}} = \frac{I(C,G)}{\sqrt{H(C)H(G)}}$
  - Value range of NMI: [0,1]
  - Value close to 1 indicates a good clustering

Pairwise Comparison-Based Methods: Jaccard Coefficient

- Pairwise comparison: treat each group in the ground truth as a class
- For each pair of objects (o_i, o_j) in D, if they are assigned to the same cluster/group, the assignment is regarded as positive; otherwise, negative
  - Depending on assignments, we have four possible cases:

Note: Total # of pairs of points

$$N = \binom{n}{2}$$

	$C(o_i) = C(o_j)$	$C(o_i) \neq C(o_j)$
$G(o_i) = G(o_j)$	true positive (TP)	false negative (FN)
$G(o_i) \neq G(o_j)$	false positive (FP)	true negative (TN)

- **Jaccard coefficient:** Ignoring the true negatives (thus asymmetric)
  - *Jaccard* = *TP*/(*TP* + *FN* + *FP*) [i.e., denominator ignores *TN*]
    - *Jaccard* = 1 if perfect clustering
- Many other measures are based on the pairwise comparison statistics:
  - Rand statistic
  - Fowlkes-Mallows measure

Intrinsic Methods (I): Dunn Index $DI = \frac{\delta}{\Delta}$

- Intrinsic methods (i.e., no ground truth) examine how compact clusters are and how well clusters are separated, based on similarity/distance measure between objects
- Dunn Index:
  - The compactness of clusters: the maximum distance between two points that belong to the same cluster: $\Delta = \max_{C(o_i)=C(o_j)}\{d(o_i, o_j)\}$
  - The degree of separation among different clusters: the minimum distance between two points that belong to different clusters: $\delta = \min_{C(o_i)\neq C(o_j)}\{d(o_i, o_j)\}$
  - The Dunn index is simply the ratio: $DI = \frac{\delta}{\Delta}$, the larger the ratio, the farther away the clusters are separated comparing to the compactness of the clusters
- Dunn index uses the extreme distances to measure the cluster compactness and inter-cluster separation and it can be affected by outliers

Intrinsic Methods (II): Silhouette Coefficient

- Suppose D is partitioned into k clusters: $C_1, \ldots, C_k$
- For each object o in D, we calculate
  - $a(o) = \frac{\sum_{o' \in C_i, o \neq o'} dist(o,o')}{|C_i|-1}$: avg distance between o and all other objects in the cluster to which o belongs, reflects the compactness of the cluster to which o belongs
  - $b(o) = \min_{C_j:1 \leq j \leq k, j \neq i} \left\{ \frac{\sum_{o' \in C_j} dist(o,o')}{|C_j|} \right\}$: minimum avg distance from o to all clusters to which o does not belong, captures the degree to which o is separated from other clusters
- Silhouette Coefficient: $s(o) = \frac{b(o)-a(o)}{\max\{a(o),b(o)\}}$, value range (-1, 1)
  - When the value of o approaches 1, the cluster containing o is compact and o is far away from other clusters, which is the preferable case
  - When the value is negative (i.e., b(o) < a(o)), o is closer to the objects in another cluster than to the objects in the same cluster as o: a bad situation to be avoided